CARE TO COMMENT ON FLOSSING?
ISSUE COMMENTS AND DISENGAGEMENT IN OPEN SOURCE

BY
COURTNEY ELTA MILLER

A Thesis

Submitted to the Division of Natural Sciences
New College of Florida
in partial fulfillment of the requirements for the degree
Bachelor of Arts
under the sponsorship of Dr. Matthew Lepinski and Dr. Andrey Skripnikov

Sarasota, Florida
May 2021

This thesis is dedicated to Chanel and Mister Rogers.

Acknowledgements

It takes a village but here are a few people who are deserving of special thanks.

Professor Doucette– Thank you for sparking my interest in computer science, being a world-class advisor, and for teaching me that I belong.

Professor Crow– Thank you for being a kind, patient, and persistent force in my mission to learn what a p-value is and for teaching me everything I know about R.

Professor Eaton– Thanks for being brave and agreeing to be on my committee. I have only known you for a year, but when you are such a passionate, energetic, and positive professor, you do not need long to impress.

Professor Skripnikov– Thank you for being one of the most effective educators I have had the privilege to work with and for being such a hands-on and helpful thesis sponsor. Your innate talent of knowing when to push, challenge, and encourage students has helped me learn more about Statistics than I ever thought I could.

Professor Lepinski– Thank you for being the embodiment of a caring ray of sunshine throughout my tenure in this department. You have been a consistently bright, empathetic, and supportive advisor. You are like the Swiss Army Knife of professors; from being an engaging lecturer, to actively involved club sponsor, to encouraging thesis advisor, you have an outstanding breadth of abilities that I have personally benefited from on countless occasions and deeply appreciate.

Bogdan and Christian – Thanks for taking a chance on me back in 2018. Dur-

ing my first academic research experience, I learned a lot from you two. The most important thing being that I love research, so thank you for that. Your continuous support over the years has been instrumental in my successes, warmed my heart, and reminded me how awesome you are. I cannot wait to keep learning from you two.

Julie Elta Miller– Whether you meant to or not, you raised a strong, resilient, and highly opinionated little girl. Thank you for giving me the space, love, and support I needed to become the person I am today.

Chanel– You may be a ten pound Toy Poodle, but the amount of joy, peace, vibrancy, and laughter you bring to my life is priceless. We've been going on walks together for nearly a decade, but your passion for following the path less traveled (and chasing squirrels) continually reminds me to keep being curious, follow my heart, and wear sneakers on walks. I could name a million other ways you enrich my life, but I will save the time and instead just say you are the most important cluster of matter in my life.

Contents

Acknowledgements **iii**

Abstract **vii**

1 Introduction **1**

2 Background and Related Work **5**

2.1 Open Source Software 5

2.1.1 Open Source Contributors 7

2.1.2 Communication in Open Source 8

2.1.3 Contributor Disengagement 9

2.2 Text Classification in Software Engineering 10

3 Phase 1 **12**

3.1 Building Issue Comment Classifier 13

3.1.1 Text Preprocessing 13

3.1.2 Creating Input Embedding 14

3.1.3 Balancing Training Data 15

3.1.4 Classifier Architecture 17

3.2 Evaluating Classifier 19

3.2.1 Experiment Setup 19

3.2.2 Results 20

4 Phase 2 **22**

4.1 Methodology 22
4.1.1 Data Collection 22
4.1.2 Data Preprocessing 25
4.1.3 Data Exploration 31
4.1.4 Model Building 36
4.2 Results 40
4.2.1 Static-Model 40
4.2.2 Non-Static-Model 41
4.2.3 Comparing Model Performances 43
4.2.4 Data Reflection 44

5 Conclusion and Future Work **46**
5.1 Threats to Validity 46
5.2 Conclusions and Future Work 47

Care to Comment on FLOSSing?

Issue Comments and Disengagement in Open Source

Courtney Elta Miller

New College of Florida, 2021

Abstract

Free/libre open source software (FLOSS) is a critical tool used as digital infrastructure for lots of modern software projects. Open source project survival is important because these projects exist in a network of interdependencies, so the failure of one project can not only hurt the other open source projects in its network, but also the other software projects that rely on them from credit bureaus to Instagram. The disengagement of open source contributors, especially core contributors, has a serious impact on the survival probability of a project. Previous research has explored various avenues of understanding disengagement including technical factors and project characteristics. However, communication, especially the content of communication is an under-explored avenue in disengagement research. In this thesis we create a classifier for issue comments and we then investigate the differences between the types of issue comments written by core contributors who disengage versus those who do not. We learned there is a significant difference in the average number of comments written across the two groups for all comment types. We also found that writing more *Action on Issue* comments, i.e., comments discussing the management of the issue thread itself, leads to a lower probability of disengagement.

Dr. Matthew Lepinski, Assistant Professor of Computer Science
April 22, 2020

Dr. Andrey Skripnikov, Assistant Professor of Statistics
April 22, 2020

Chapter 1

Introduction

Free/libre open source software (FLOSS) is software whose source code is freely distributable and modifiable. We will be referring to it as open source for short. From Google to roughly 70% of all websites[1], lots of people use open source. Open source is like the digital infrastructure modern software is built on. It allows modern software to speed along on highways rather than having to plow their own dirt roads.

Despite the importance of open source, most open source projects fail [36]. One common reasons why these projects fail is developer disengagement [13]. A post-mortem of depreciated open source projects found that 41% cited issues related to the sustained participation of contributors, i.e., loss of time or interest of contributors [10]. Another study found that 80% of open source projects fail because of issues related to contributor disengagement [54]. In this thesis, we focus on understanding the connection between contributor communications and disengagement.

Many open source projects rely on a small number of 'core' contributors who reside at the heart of these projects to organize, manage, and run them. Oftentimes, the number of core contributors is quite small, commonly as few as one or two [4]. The disengagement of these core contributors can be particularly harmful [20]. Even if a project survives the disengagement of these contributors, they may still face serious side effects from this disengagement including expensive knowledge loss [32], schedule delays [60], reduced software quality [23], and reduced productivity [62].

[1] https://news.netcraft.com/archives/category/web-server-survey/

Developer disengagement poses a substantial threat to both project-level sustainability, and through cascading effects, ecosystem-level sustainability [59]. The failure of a single project in an ecosystem can threaten the ecosystem itself because of project interdependencies. So even if a less popular project fails, if a more popular one relied on that now depreciated project, that can cause serious problems for the more popular project as well as the rest of the projects in the ecosystem's interdependency network.

Because of this, there has been lots of work done studying developer disengagement. A good deal of the research on developer disengagement has focused on individual technical factors and project characteristics. Some of these individual technical factors include tenure on a project [55, 40], volume of code [14, 40, 63], and the type of contributions made [14, 20] (e.g., code, documentation, discussion, etc.). Some of the project-level characteristics that have been studied include a project's relative position in its dependency network and the number of project ties [59].

We postulate that the impact of communication is an underexplored area of open source disengagement research. While there has been some research considering the frequency of communications [14], or the impact of social capital on sustained participation [51], there has been little research exploring the relationship between contributor communications and disengagement.

Communication is a big challenge within the development process [5, 44]. As such, studying the relationship between developer communications and various aspects of the open source process has received a fair amount of attention. Some examples of this research include how the politeness of responses to new contributor questions affects whether they stick around [57], and which developer personality traits, studied through code commits and mailing lists, suggest a higher likelihood they will become a project contributor [7]. However, this sort of exploration of the content of communication had been applied less within the context of studying open source disengagement specifically.

This thesis aims to take a first step at addressing this gap in the research by utilizing the contents of the rich corpus of text data from GitHub to explore the

relationship between contributor communications and disengagement. In particular, we focus on Issue Tracking Systems (ITS) like GitHub's Issues feature which allow developers to have asynchronous conversations about various tasks or issues related to project development. These issues can include testing, bugs, requests for new features, or documentation questions. Through these issue threads, developers and project users engage in often in-depth discussions that cover both technical and non-technical discussions.

These issue threads contain a wide variety of information types as classified by Arya *et al.* including motivations for addressing an issue, social conversations, commitments regarding contributions, and requests for additional support [3]. We hypothesize that by understanding the types of issue comments a contributor is writing and the patterns of those comments over time, we will gain useful insights towards understanding and ultimately preventing disengagement.

To this end, we adopt the issue comment type categories and classification framework of Arya *et al.* We employ a two-phase methodology described below:

Phase 1. We first design, implement, and evaluate a text classifier for issue comments, building off of the original architecture designed by Arya *et al.* [3]. We make some modifications to the architecture that we hypothesize will increase performance and compare the results to those of the original architecture. In particular, we address class imbalance in the training data differently and use an alternative mechanism for creating sentence-level embeddings. These sentence-level embeddings are the input to the classifier.

Phase 2. We then use the model built in Phase 1 to classify the issue comments for a stratified sample of engaged and disengaged core open source contributors. We compare the frequency of comment types across the groups. We then create aggregate temporal variables to encapsulate the temporal dynamic of each issue comment type's usage. Finally, we build two models to categorize users based on disengagement using issue comments. One model represents the issue comments using their total count, and the other uses the aggregate temporal variables we created.

In Phase 1 we learned our updated architecture did not outperform the original

classifier's overall performance. However, our classifier did outperform the original model on three of the thirteen classes.

In Phase 2, we confirmed a significant difference in the total number of comments made across the groups for each comment type. Moreover, as part of a sanity check, we verified that the number of disengaged contributor comments decreases at a significantly faster pace over time than those of engaged contributors. Additionally, we found in both models that contributing more *Action on Issue* comments leads to lower probability of disengagement. *Action on Issue* comments refer to discussions of appropriate handling of the issue comment itself, e.g., the closing of the issue comment. We also found that our second model that accounted for the temporal dynamic of the issue comments slightly outperformed the first model that just considered total counts.

In this paper we contribute (1) the updated design and evaluation of a modified supervised text classifier for issue comments; (2) a refined definition for core contributor disengagement; (3) a comparison of the patterns and frequency of issue comments written by disengaged and engaged contributors; and finally (4) two models classifying disengagement using issue comment types.

The structure of the rest of the paper is as follows, in Chapter 2 we describe the background and related work, in Chapter 3 we discuss Phase 1, in Chapter 4 we discuss Phase 2, and in Chapter 5 we discuss conclusions, future work, and threats to validity.

Chapter 2

Background and Related Work

2.1 Open Source Software

What is Open Source Software? Free/libre open source software (FLOSS) is software whose source code is freely distributable, accessible, and modifiable to users. For example, lets pretend this thesis is being read on a PDF reader. If that PDF reader is Adobe Acrobat DC, the user does not have access to the source code or the ability to freely distribute the software therefore it is *closed source*. However, if you are using LibreOffice, then the user does have access to the source code and the ability to freely distribute the software therefore it is free/libre open source or what we will refer to as just *open source* for short throughout this thesis.

Since open source software is freely distributed and modifiable, it is used as a critical building block for many software projects. By utilizing open source frameworks, developers can avoid repetitive foundational development work and instead focus on producing more rapid innovation and bringing their ideas to life. A common analogy within the software engineering community is that open source is the infrastructure that modern software projects are built on, like the roads and bridges utilized by commuters to get from point A to point B [18].

What is GitHub? Before we can explain what GitHub is, we must first briefly explain what version control and Git are. Version control allows developers to track and monitor changes to code throughout the development process. Git is a version

control software that allows for easier collaboration through the use of branches, which are essentially different versions of the same code base so developers can work simultaneously on a project and merge their progress together into a main branch.

GitHub is a Git version control website used by software developers to host software projects. There are other Git version control websites including Bitbucket and GitLab. We study open source using GitHub specifically because it is currently the most popular option and it has the most active developers on it.

Issues are a feature of GitHub, they are essentially threads of conversations within a repository that are used to "keep track of tasks, enhancements, and bugs" on a given project[1]. Throughout this thesis, we will refer to an issue thread in its entirety as an *issue* or *issue thread*, and a single comment within an *issue thread* as an *issue comment*.

Why is Open Source Important? Top caliber software projects utilize open source. Instagram has a page where they "thank the open source community for all their contributions" and list the specific libraries they use[2]. Netflix[3], Google[4], and Facebook[5] are all open source users and supporters.

In 2018 Microsoft even acquired GitHub, which at the time had more than 28 million developer users and as of September 2020 had 56 million developer users[6]. As Satya Nadella, CEO of Microsoft, put it in a blog post discussing the acquisition "the real power comes when every developer can create together, collaborate, share code and build on each other's work."[7]

Because of the importance of open source, it has been been studied by researchers for decades [21]. Within the overarching category of open source, previous research has explored a vast number of topics. A sampling of these topics include how social capital effects sustained participation [51], the evolution of forking on GitHub [64], donation systems in open source [48], how developers evaluate pull requests [22], the

[1]https://guides.github.com/features/issues/
[2]https://www.instagram.com/about/legal/libraries/
[3]https://netflix.github.io
[4]https://opensource.google
[5]https://opensource.facebook.com
[6]https://octoverse.github.com
[7]https://blogs.microsoft.com/blog/2018/06/04/microsoft-github-empowering-developers/

serious lack of diversity in open source [2], using open source in pedagogy [49], and the motivations of contributors to open source for social good (OSS4SG) [31].

2.1.1 Open Source Contributors

Open source contributors are developers who contribute to open source projects and open source ecosystems at large. As with any software project, contributors are at the heart of open source. Without open source contributors, there would be no open source. As such, there has been a great deal of research done on open source contributors [52, 19, 50, 8, 30].

Within the context of studying open source contributors, there have been a wide array of avenues traveled in an attempt to improve our understanding of them. A sampling of these topics include challenges contributors face when on-boarding onto projects [29, 56], studying which new contributors are likely to continue contributing to a project [54], and the motivations of contributors to open source [37].

However, not all open source contributors perform the same tasks. Within a given project, there are often lots of moving parts and subsequently different kinds of contributors who have various roles and responsibilities.

There are different theories about the structure of open source projects and the contributors within them, but one popular theory is the onion model [47]. The onion model suggests that there are a small number of core contributors at the center of a project who organize, run, and manage the project [15]. Then moving away from the core there are increasingly peripheral layers of contributors with various roles, known as peripheral contributors, who have less responsibility and fewer contributions [35].

While all open source contributors provide valuable contributions to a project, not all contributions have the same impact. It is important to emphasize that as described in the onion model, on many projects a small number of core contributors are responsible for essentially keeping these projects alive and progressing. A study of 133 popular open source projects hosted on GitHub found that 65% of projects depend on one or two core contributors to keep their projects alive [4].

2.1.2 Communication in Open Source

One of the most challenging aspects of software development is communication [5]. Coordinating organizational plans, conveying ideas about bug fixes, and discussing code reviews are just a small sample of the challenging communication tasks involved with development.

Code reviews are the review process where peers certify that a patch, new feature, or other addition to a code base is in good working order. Code reviews are notorious in software engineering for being an example of a communication challenge that can lead to negative feelings and frustration[8]. A study of developers at Google found that experiencing pushback in communications from reviewers during code review can lead to frustration and negative experiences of developers [17]. Negative experiences and emotions caused by the development process can also pose long-term issues. A survey of software developers found that feelings of unhappiness arising during the development process can lead to low motivation and withdrawal from work [28].

Subsequently, as with all other development work, communication during the open source development process can have a critical impact on the project itself. Some of the most commonly cited challenges by open source contributors are related to communication. Suboptimal responsiveness of maintainers and issues communicating the rational for proposed changes are specifically cited as tough challenges faced frequently [27].

One avenue of previous research has involved mining text from various forms of communication within open source projects including issue managers and mailing lists to learn valuable insights about open source contributors and the projects themselves.

A study of comments committed by developers on 22 open source projects found that the politeness of the comments had an effect on how long it took to fix issues, with the issues on projects with more polite comments being fixed faster [16]. Calefato *et al.* used text from the websites, mailing list archives, and code repositories of projects as input into the *Personality Insights* API and found that "stronger openness and agreeableness traits scores are associated with higher chances for developers to be-

[8]https://medium.com/idyllic-geeks/code-reviews-do-not-have-to-be-stressful-919e0a8377a1

come a project contributor [7]." Additionally, a study of newcomers on the Hadoop Common project found evidence that the decision of newcomers to disengage from a project was influenced by the responses to their first interaction on a project, specifically the response itself and it's qualities, i.e., was the answer helpful, not useful, polite, or indifferent [57].

We expand upon this research direction by mining and classifying core contributor issue comments and examining how the *types* of issue comments made relate to disengagement, which we elaborate on next.

2.1.3 Contributor Disengagement

Contributor engagement is critical to the success of open source projects, and their disengagement is a big problem [32]. Schilling *et al.* found that roughly 80% of open source projects failed due to"lacking sustained contributors [54]."

In particular, we know core contributors are at the heart of many open source projects so their disengagement can be particularly harmful to these projects [20]. Since core contributors are often responsible for managing and organizing the project, their disengagement can cause very expensive knowledge loss [32], reduce software quality [23], and significantly reduce project productivity [62]. Coelho *et al.* surveyed maintainers of depreciated open source projects and found that two of most commonly cited reasons for project failure were lack of time and lack of interest of core contributors [10].

Because of the dangers and costly side effects of poor contributor retention, contributor disengagement has been studied in depth. Much of the previous research on disengagement has focused on factors that relate to either a contributors tenure on a project [55], or technical contributions [40, 45]. Miller *et al.* found that having higher overall levels of activity decrease a core contributor's risk of disengagement [45].

In Section 2.1.2 we established that communication is critical to the development process and that communication in open source has been studied in depth. However, under the umbrella of communication in open source, there has been limited work studying the impact of communications on contributor disengagement. Qiu *et al.*

found that social capital "positively affects sustained participated in open source on GitHub [51]." Previous research has also found that sustained participants often contribute to a project both in terms of mentoring others and developing code [20].

Of studies that explore communication factors, many focus on summary characteristics like frequency of communication. Constantinou *et al.* studied the relationship between the frequency of issue and pull request comments and disengagement within the RubyGems and npm ecosystems. They found that developers who have a higher probability of disengaging communicate less frequently [14].

We are interested in expanding this research by investigating the *content of issue comments* and their relationship to *core contributor disengagement*.

2.2 Text Classification in Software Engineering

Within the software engineering domain, natural language processing (NLP) has been used to classify text in a wide array of applications [41, 16, 57, 7]

However, we focus our discussion on the research by Arya *et al.* which serves as the basis of our replication work.

This thesis work builds upon an important paper by Arya *et al.* which addressed the challenge of identifying and accessing the rich corpus of information about a software project often embedded in issue threads [3].

Their two pronged goal was to first understand what *types* of information are present in the issue threads and secondly to create a supervised text classifier to automate the classification process. The hope was that once the different types of comments in a thread could be identified, information could be easily parsed to address the interests and needs of various open source stakeholders since issue threads are often long and complicated, making them difficult to parse manually. Figuratively, imagine a tool that acted like CMD+F but for comment types in an issue thread.

To identify the types of comments present, they used qualitative content analysis to manually label 4656 comment sentences from 15 issue threads leading to the creation of 16 issue comment types. Examples of the comment types created include

Observed Bug Behavior, *Solution Discussion*, and *Action on Issue*. The full code book with detailed explanations of these comment types can be find in the original paper [3].

To create an automated classifier for issue comments they experimented with 12 configurations of the model including the type of model used (logistic regression or random forest), the feature set used (textual, conversational, or both), and how class imbalance was addressed (SMOTE or class weight). They evaluated these configurations on two use cases. Scenario 1 was labeling comments from an issue thread that had already been partially seen, Scenario 2 was labeling comments from a completely new issue thread.

We build upon this research with a slightly different goal in mind. Instead of labeling all the comments in a given issue thread, we are instead interested in labeling all of the issue comments for a given contributor (regardless of what issue thread they originate from). We are specifically interested in looking at the issue comments for a group of engaged and disengaged contributors so we can investigate the differences between the types of issue comments written by contributors who disengage versus those who do not. Since we are interested in labeling issue comments from new issue threads, in Phase 1 we build off the model that performed best for Scenario 2 in the previous paper.

In the original paper, the model that performed best in Scenario 2 was a logistic regression model with textual input features that used class weight adjustment to address class imbalance. We will refer to this model as the Previous-Model. For more details on the implementation of the Previous-Model, please refer to the original paper [3].

Chapter 3

Phase 1

As discussed in Section 2.2, Arya *et al.* considered two scenarios, and we are interested replicating the model that performed best in Scenario 2, which we refer to as the Previous-Model.

We plan to build off of the Previous-Model to create what we will call the Updated-Model. While there will be a small number of changes to the Updated-Model architecture, we will still use the same data preprocessing, classification layer, training data, and evaluation metrics as used in the Previous-Model for ease of comparison. There were two major changes made to the Updated-Model.

First, SMOTE upsampling was used to address class imbalance instead of adjusting class weight. The training data contains 4656 comment sentences which makes it a relatively small data set by machine learning standards. In particular because the class imbalance is moderate, we believe Synthetic Minority Over-Sampling Technique (SMOTE) is a good option because it creates artificial cases of the minority classes which addresses the class imbalance while also filling out the training data a little. We discuss SMOTE and the issue of class imbalance in further detail in Section 3.1.3.

The second major change was our method for creating numeric representations of the comments which act as input for the classifier. The Previous-Model used Term Frequency-Inverse Document Frequency (TF-IDF) to create numerical representations of sentences. To understand TF-IDF let us first consider the Bag of Words (BoW) technique which creates a numerical representation for a sentence by counting

the number of times each word in the vocabulary of the corpus appears in a sentence. The main issue of BoW that TF-IDF addresses is that words that appear frequently like *the* or *it* do not necessarily have the most value or should be weighed most heavily. TF-IDF takes the frequency of words in a sentence and multiplies it by the inverse of the number of sentences that word appears in. For example, this makes the TF-IDF score for a word like *the* very low because it appears in most sentences.

While TF-IDF is a marked improvement on BoW, it does not have a way of representing semantic similarities between words which we feel is very important information when creating a sentence embedding and capturing the essence of a sentence. Because of this, we used Word2Vec and an LSTM model to create our sentence level embeddings. We go into further detail on Word2Vec and LSTM in Section 3.1.2 and Section 3.1.4 respectively.

In Phase 1 we begin by performing text preprocessing, followed by creating the input embeddings, describing the modified model architecture, and finally comparing the Updated-Model performance to the Previous-Model.

3.1 Building Issue Comment Classifier

3.1.1 Text Preprocessing

We began by pulling the replication data from the replication package on GitHub [1]. We followed the same text preprocessing procedure as done in the original paper, with the addition of one extra step. The additional step is that we replaced the '+1' symbol with the token PLUS ONE. We describe in detail the text preprocessing steps below which were accomplished using regular expressions in Python.

- Replace '+1' with the token PLUS ONE (this is the amendment we made to the preprocessing procedure because we observed '+1' being used a lot in the comments and since it could be an important affirmative remark in the

[1] https://git.io/fhQTt

context of a comment, we wanted to be sure they were included in the lexicon accordingly).

- Replace code chunks with the token CODE (note: this preprocessing step was already included in the replication data pulled from the repo)
- Replace @ mentions to contributors with the token SCREEN NAME
- Replace quotations with the token QUOTE
- Replace hyperlinks with the token URL

We then tokenized the text, replaced common contractions, removed all punctuation, made all the text lower case, and lemmatized the text. Lemmatizing the text means reducing words down to their lemma or base forms. For example, the verbs *running*, *runs*, or *ran* would be replaced with their lemma *run*.

3.1.2 Creating Input Embedding

Picking up where we left off in Section 3.1.1, each sentence is currently a list of stemmed tokens which will now be transformed into numerical vector representation using Word2Vec word embeddings [43].

Effective word embeddings should capture (1) the semantic similarity between words in the lexicon; (2) the relation between words; and (3) the context of a word within the document.

Generally speaking, Word2Vec uses a shallow neural network to create vector embeddings for words so that words that are more similar like *run* and *jog* are closer together in the *n-dimensional* space made where n is the size of the vector embeddings, and words that are less similar like *lychee* and *poodle* are farther apart.

Training effective Word2Vec vector embeddings traditionally requires training on a corpus of millions and millions of words, which is slightly outside the scope of this undergraduate thesis project.

Instead, we used a pre-trained Work2Vec model of word embedding vectors made by the creators of Word2Vec themselves, Google [2]. This Word2Vec model of pre-trained vectors was trained on a subset of the Google News dataset with a corpus of roughly 3 million words and phrases [3].

For each word in a sentence, we search for the word's vector embedding in the Google Word2Vec model. If a word is not present in the Google Word2Vec model lexicon, we replaced it with the average vector from the model. Now, each sentence which was previously a list of stemmed tokens is a list of vectors (there is a vector for each of the stemmed tokens, i.e., words). These list of vectors are our word-level embedding that will be the input to our model. Figure 3.2 illustrates the process of creating the input embeddings.

3.1.3 Balancing Training Data

The training data from the replication packet consists of 13 classes, but they are not all equally represented. When the classes in a data set are not equally represented, that is referred to as class imbalance [46]. If the class imbalance in a data set is not properly addressed, it can have serious negative effects on the models trained on said data [34, 58]. Figure 3.1 is a visualization of the class imbalance present in the training data.

Of the 4330 annotated sentences in the training data, the most common label, *Solution Discussion*, represents roughly a quarter of the data set with 1411 sentences. Whereas the least common label, *Action on Issue*, only constitutes 61 or 1% of the comments which is classified as moderate imbalance [4].

Class imbalance can cause problems in many classification algorithms. For example, imagine we are trying to classify pull requests as either accepted or rejected and in the data set there are 90 rejected and 10 accepted pull requests. The classification algorithm could reach 90% accuracy by predicting all pull requests are rejected. A

[2]Thomas Mikolov developed the Word2Vec technique in 2013 while working at Google.

[3]https://code.google.com/archive/p/word2vec/

[4]https://developers.google.com/machine-learning/data-prep/construct/sampling-splitting/imbalanced-data

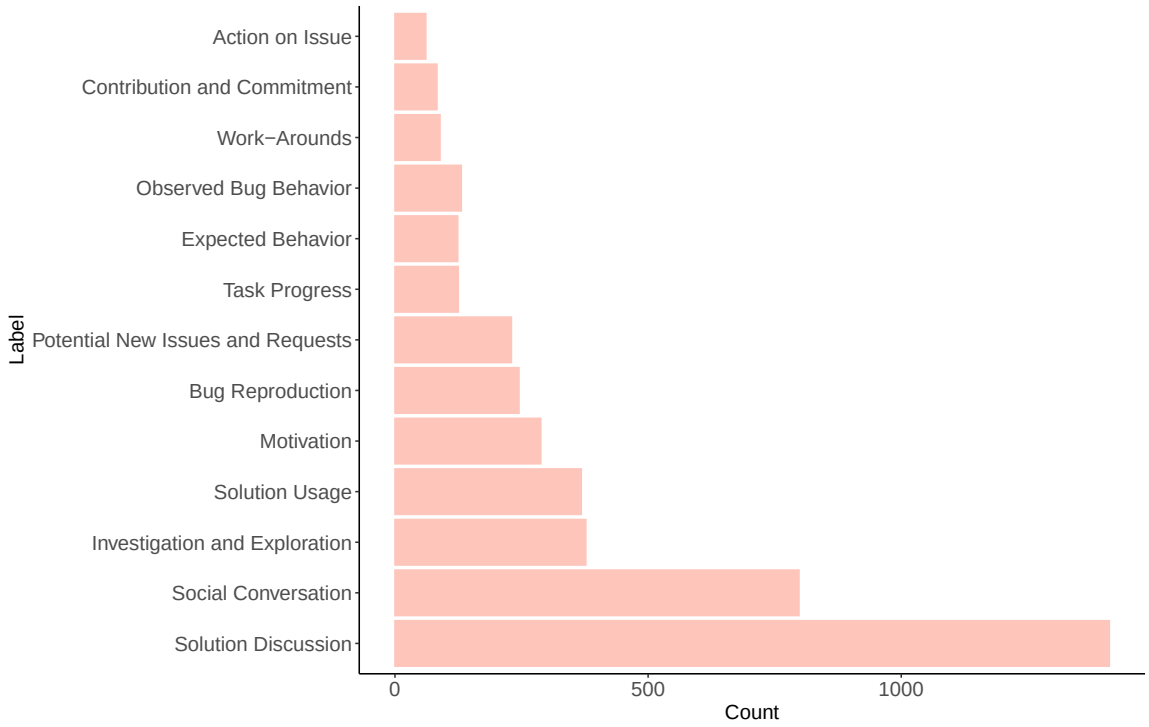


Figure 3.1: Distribution of cases among the labels

solution with such a high accuracy would be appealing to many classification algorithms and their underlying optimization process. In fact, it would be very difficult to yield continued improvement past this point since using an accuracy-based loss function, if the algorithm were to correctly label 10% of the minority class, for the overall solution to appear like an improvement it would have to evade introducing even a 1% error in the majority class. It should also be noted that while class imbalance can have a negative impact on many classification algorithms, the severity of the practical impact is dependent on the intended use case for the classification model being constructed. For example in our case, since we care about observing the types of issue comments made by different kinds of contributors, misclassification especially of minority classes is problematic and why we address class imbalance.

There are two main ways of addressing class imbalance in a data set [34], (1) by directly optimizing the machine learning algorithm to account for the class imbalance; or (2) by performing data preprocessing that changes the actual makeup of the data so the classes are balanced.

The Previous-Model addressed class imbalance using the former technique by adjusting the class weights in their model. By increasing the weight of minority classes, it is no longer in the classifiers best interest to always choose the majority class since there is a noticeable penalty for missing cases from minority classes.

Due to the relatively small sample size and moderate class imbalance, we chose to go with an upsampling data preprocessing technique instead. Specifically, we used the Synthetic Minority Oversampling Technique (SMOTE) [9]. SMOTE works by generating new synthetic cases for the minority classes. These synthetic cases are linear interpolations between existing cases for a given minority class. Since we have created numerical vector representations for each sentence, the new synthetic cases can be considered averages between two given cases. SMOTE is only performed on the training data, not the test data.

3.1.4 Classifier Architecture

The input embedding created in Section 3.1.2 is a word-level embedding. Meaning it has not yet been pooled together to create a sentence-level embedding, i.e., sentence representation, which will ultimately be the input to the linear layer of the model. This was done intentionally. A common procedure when using Word2Vec is to pool together or essentially create an average representation of all the words in a sentence. However, we wanted to make a model that would learn to create better sentence-level embeddings using its own errors.

We designed a neural network architecture that includes a linear layer and a log softmax layer which together do the same job as the logistic regression model in the Previous-Model. In addition, we added an LSTM layer to the beginning of the model. The LSTM layer takes in the input embedding and transforms it into a sentence-level embedding which then becomes the input to the linear layer as previously discussed. Importantly, the error from the classification is back propagated to the LSTM layer so the LSTM layer can learn to make more 'helpful' sentence-level embeddings. Figure 3.2 illustrates the procedure for Phase 1 including the architecture for this classifier. We implemented our classifier using PyTorch and the nn.module.

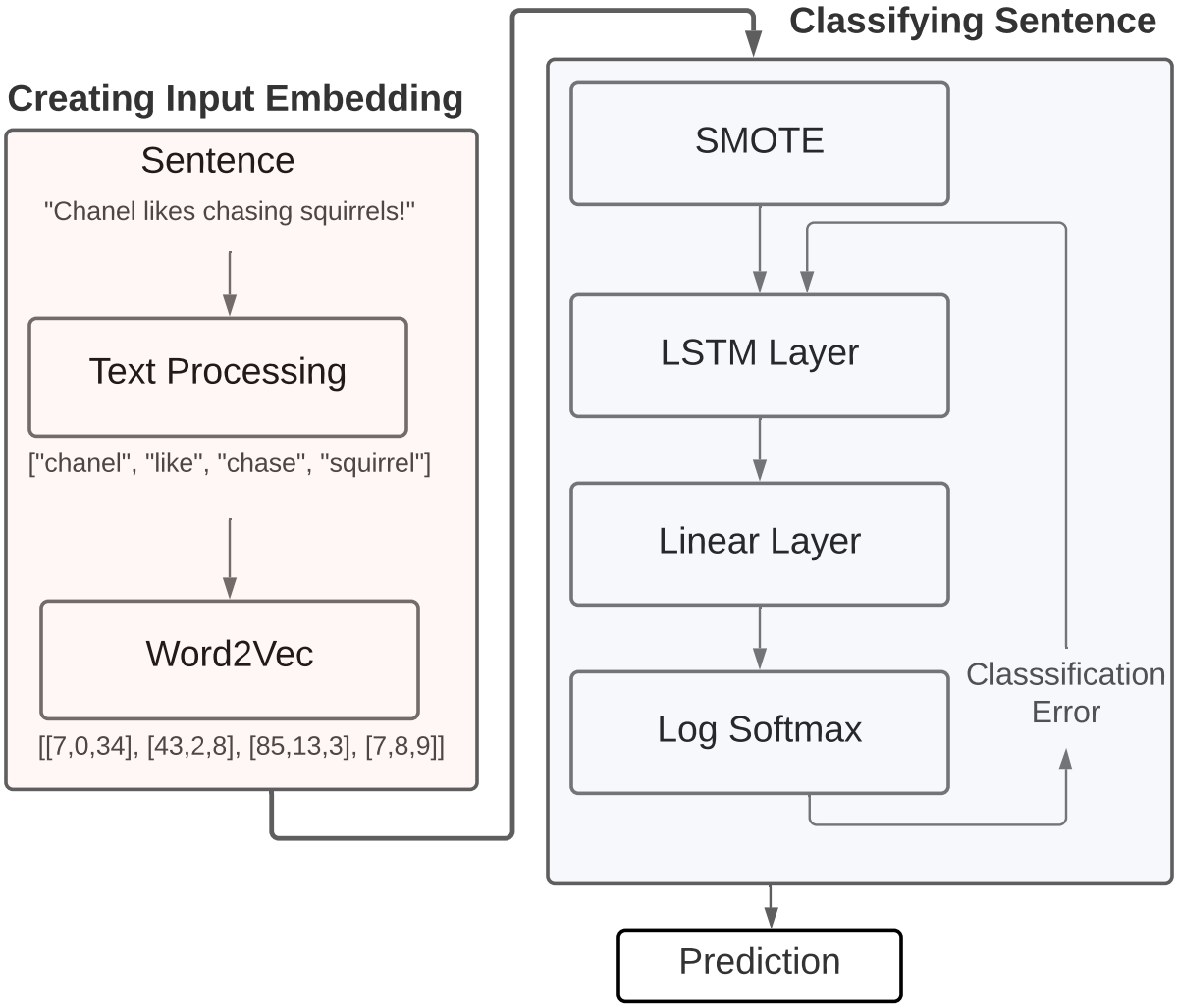


Figure 3.2: Flowchart of Phase 1 Methodology

For our loss function we used negative log likelihood[5]. Negative log likelihood is a cost function that tells us how well our model has performed, with lower values indicating better performance.

When training a neural network or other machine learning model, one important consideration is the number of epochs used. An epoch represents a full pass through the training data. The number of epochs performed can have a significant impact on model performance [25].

[5]https://pytorch.org/docs/stable/generated/torch.nn.NLLLoss.html

To determine how many epochs we should use, we implemented a binary-search-like-process ranging from 15 to 200 epochs. For each number of epochs evaluated, X, we ran the model three times using X epochs and recorded the performance. Ultimately, the models that ran 25 epochs had the 'best' performance.

3.2 Evaluating Classifier

3.2.1 Experiment Setup

We followed the same experiment setup as was used with the Previous-Model.

There are 15 issue threads in the training data which originate from three popular Articicial Intelligence projects on GitHub; TensorFlow[6], scikit-learn[7], and spaCy[8].

We used leave one out cross validation (LOOCV) for our model evaluation [53]. LOOCV involves using one of the training cases to evaluate the model, and training on the rest. In this circumstance, that means we will leave out one of the issue threads for the testing set and use the remaining 14 threads as the training data. We do this process once for each of the issue threads, meaning in total we will run the model 15 times.

To evaluate the performance of our model, we used the same metrics that were used for the Previous-Model which are precision, recall and F1-Score.

Precision is the percentage of cases that were labeled as belonging to a given class that actually belonged to said class. Formally, let *tp* be the number of true positives, meaning the number of cases belonging to a class that were correctly labeled as such, and let *fp* be the number of false positives, or the number of cases that were labeled as belonging to a class that actually did not. Then, we can formally define the precision of a model, P, as being $P = \frac{tp}{tp+fp}$. Recall is the percentage of cases belonging to a class that were correctly labeled as such. Formally, let *fn* be the number of false positives, meaning the number of cases of a class that were labeled as belonging to a

[6]https://github.com/tensorflow/tensorflow
[7]https://github.com/scikit-learn/scikit-learn
[8]https://github.com/explosion/spaCy

different class. Then we can define the recall of a model, R, to be $R = \frac{tp}{tp+fn}$.

One drawback of using precision and recall is that they have a symmetric relationship [34, 6]. Generally speaking, you can increase the precision of a model by reducing its recall and vise versa. This is can often be accomplished with minor tuning of things like the classification thresholds of a model. However, when abstractly considering the evaluation of classification models there is not a general consensus on which is more important; avoiding false positives or false negatives. This means when using precision and recall, there is a certain amount of domain and use case specific subjectivity surrounding which one to use and how to weight the importance of each. Fortunately, this is where F1-Scores come in. F1-Scores serve as a pleasant middle ground between precision and recall. F1-Scores are maximized when precision and recall are both high, and are minimized when there is a large disparity between the two. Formally, the F1-score of a model is defined as $F = \frac{2PR}{P+R}$.

3.2.2 Results

The Updated-Model achieved a weighted average F1-score of 0.3 compared to the Previous-Model's F1-score of 0.42. Table 3.1 compares the performance of both models, and it can be seen that generally speaking the Previous-Model significantly outperforms the Updated-Model.

While the Previous-Model generally outperforms the Updated-Model, there are a small number of classes in which the Updated-Model does outperform the Previous-Model. Specifically if we look at the F1-Scores, we can see the Previous-Model performs better on the *Observed Bug Behavior*, *Potential New Issues and Requests*, and *Solution Usage*. Interestingly, all three of these classes are roughly in the middle of the pack in terms of class size, with 131, 230, and 368 comments respectively. In fact, *Potential New Issues and Requests* is the median class size with 230 comments.

Given that the general performance of the Updated-Model was so much lower, the fact that it only outperformed the Previous-Model on middle of the pack classes leads to questions and speculation about what components of the Updated-Model might have influenced this.

Label	Precision		Recall		F1-Score		Support	
Expected Behavior	0.12	0.71	0.06	0.10	0.07	0.15	123	124
Motivation	0.19	0.44	0.08	0.10	0.09	0.13	286	288
Observed Bug Behavior	0.28	0.23	0.07	0.03	0.11	0.04	130	131
Bug Reproduction	0.30	0.53	0.24	0.36	0.26	0.42	245	245
Investigation and Exploration	0.39	0.47	0.07	0.24	0.11	0.31	376	377
Solution Discussion	0.54	0.59	0.35	0.65	0.39	0.58	1407	1411
Contribution and Commitment	0.11	0.51	0.17	0.31	0.12	0.37	83	83
Task Progress	0.17	0.35	0.16	0.26	0.16	0.29	125	125
Potential New Issues and Requests	0.26	0.10	0.04	0.03	0.06	0.03	228	230
Solution Usage	0.57	0.57	0.12	0.08	0.16	0.12	366	368
Workarounds	0.05	0.51	0.05	0.06	0.04	0.09	89	89
Action on Issue	0.32	0.78	0.38	0.49	0.33	0.58	61	61
Social Conversation	0.61	0.74	0.54	0.69	0.57	0.70	775	798
Weighted average/Total	0.44	0.55	0.27	0.42	0.30	0.42	4294	4330

Table 3.1: Comparing the performance of the Previous-Model (blue) and Updated-Model (white)

Ultimately the key differences between the Previous-Model and Updated-Model were (1) the way we addressed class imbalance; and (2) the input embeddings for the logistic regression, so explorations on these findings should consider starting with those two factors.

Chapter 4

Phase 2

4.1 Methodology

4.1.1 Data Collection

Defining and Identifying Core Disengaged Contributors. We are interested in comparing the types of issue comments written by disengaged versus engaged core contributors. We are interested in observing if there are unique patterns that could be used to signal disengagement. We must first formally define what constitutes a 'core' contributor and their 'disengagement'.

We build off of and refine the definitions used by Miller et al. [45]. In this previous work, disengaged core contributors were defined as those "who contributed at least 100 commits per six-month period for three consecutive periods, but at most 5 commits in the following period (the five commit threshold allows for some residual activity) [45]."

We wanted to create a more granular measure so to this end we used eight three-month windows, P1-P8, rather than four six-month windows. Additionally we wished to create a more dynamic metric so we could capture a more varied array of disengagement patterns. Instead of using numerical thresholds to signal disengagement, we used proportional changes in commit count to allow for more flexibility. For example, if there was a contributor, Julie, who normally contributed 500 commits per time window then dropped down to only doing 10, by the original definition using a numer-

ical threshold Julie has not disengaged. But if you consider the sharp proportional decrease, then Julie's commit pattern could in fact be considered disengagement.

Next we describe in detail our definition of core contributor disengagement.

For the the first six consecutive periods, P1-P6, they must contribute at least 50 commits. The first six periods establish they are core contributors. In the next consecutive period, P7, they must contribute *at most* 50% the number of commits they did in P6. Meaning in P7 they must have at least a 50% decrease in number of commits from the previous period. Similarly in P8, they must contribute *at most* 50% the number of commits they did in P7.

Lastly, in P8 they must contribute less than 5% of their average number of commits from the first six period when they were engaged. We used this last condition as a threshold to ensure contributors had a significant decrease without using a numeric threshold.

The engaged core contributors for the control group were defined as having at least 50 commits in all eight periods. Figure 4.1 provides a visual representation of the definitions for core engaged and disengaged contributors.

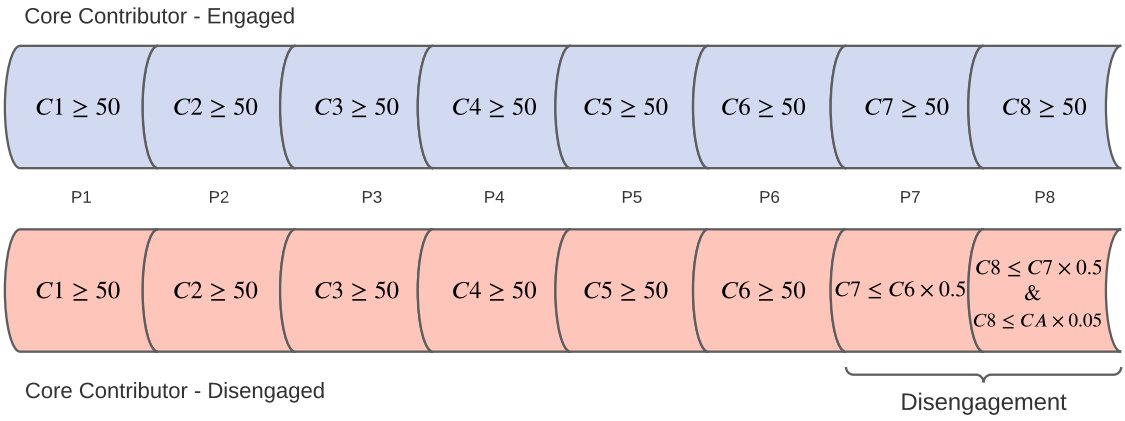


Figure 4.1: Definition of core contributor disengagement
Let $C\lambda$ be the number of commits in period $P\lambda$ where $\lambda \in 1, 2, 3, 4, 5, 6, 7, 8$
And let CA be the average number of commits in the first six periods, $P1 : P6$

Control Variables. From previous literature we know there are factors that can be useful when modeling disengagement, and that these factors can be operationalized

using publicly available trace data from GitHub [45]. Below we describe the control factors from previous literature which we will include in our models:

- Organizational Affiliation - Whether the contributor has an organization affiliation listed on their public profile.
- Activity - The total sum of their activity on GitHub during a given time period including commits as well as support work (i.e., issue comments, pull request comments, etc.,)
- Average Team Size - The average number of contributors on their main projects. To find their 'main projects' we sorted the projects they contributed to in descending order based on contribution levels. Major projects were defined as as the top projects that constitute at least 50% of their commits.
- Average Number of Stars - The average number of stars on their main projects, see *Average Team Size*.
- Percent95 - The percent of all their commits that are made between the hours of 07:00 and 19:00.
- PercentSupport - The percent of all their activity that is support work as opposed to commits.

We used GHTorrent [26] to identify a sample of engaged and disengaged core contributors. Their issue comments and other control data was also collected using GHTorrent.

Since engaged contributors had a more broad definition, there were enough candidates to select a random sample. For disengaged contributors, we pulled all contributors who matched the definition during the time period we measured which began in March 2015. We ended up with a sample comprised of 243 engaged contributors and 231 disengaged contributors, with a total sample size of 474.

4.1.2 Data Preprocessing

Preliminary Visualization of Comment Type Counts

We began by getting a general sense of the data and what the longitudinal patterns look like for each issue comment type. To this end we created a line plot for each issue comment type, where each case is a contributor. We examined how their counts have changed over time. This also allowed us to informally identify any outliers, of which there were a small number. We also created a plot for each comment type with the average lines for both groups so we could compare the patterns over time for disengaged versus engaged contributors. Figure 4.2 shows and example of these plots for *Social Conversation* comments. As expected, we do not see much decrease for engaged contributors, while there is a clear drop-off towards the latter periods for disengaged contributors, in large part due to how we defined the disengagement metrics.

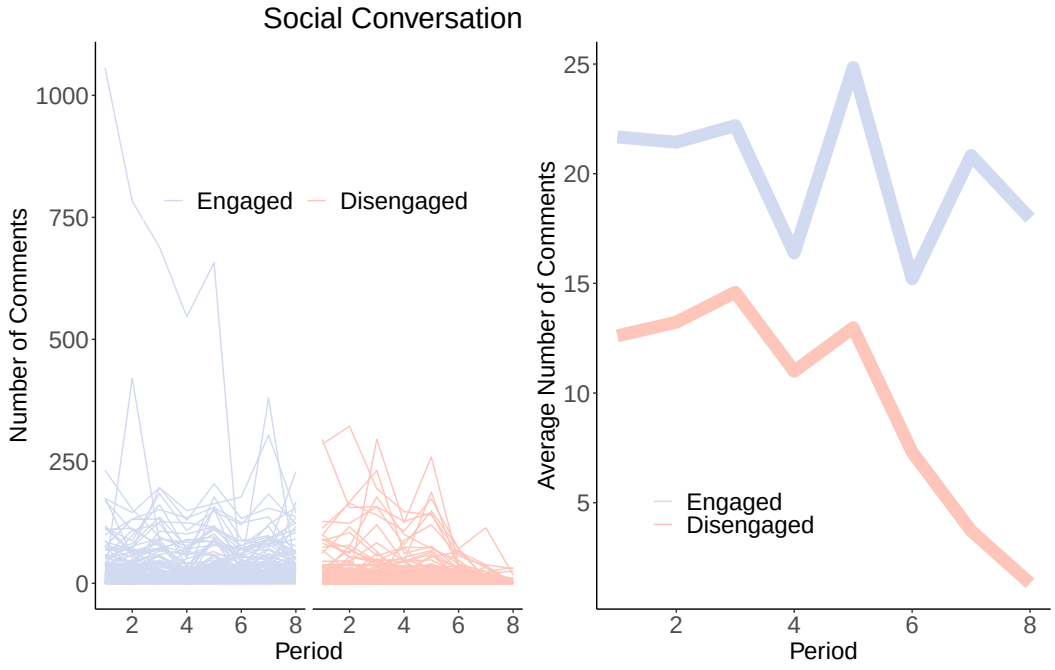


Figure 4.2: Examples of exploratory longitudinal line plot using all contributors (left) and average (right) for *Social Conversation* comments

Addressing Outliers. When creating the initial line plots, we came across some clear visual outliers that warranted further exploration. An example outlier can be

seen in the left plot in Figure 4.2, where in the engaged (blue) group there is an outlier that has an initial value over double that of any of the other contributors. We identified six contributors who were visual outliers. For each of these contributors, we examined the content of their issue comments during the periods of time where there behavior classified them as an outlier. We checked to determine if they appeared to be contributing normal comments at a high frequency, or if the comments appeared to be spam. Out of the six contributors who were flagged, only one appeared to be generating spam comments. This one contributor was removed from the pool, while the rest remained in the sample since their comments appeared normal.

Checking for Linear Correlation Among Comment Types

Before we began modeling, we first wanted to examine the full time series dynamics among the issue types, and ensure there were no strong correlations, which could cause introduce collinearity issues for our regression models. In regression, collinearity is when there is a meaningful linear relationship between predictor factors, i.e., two predictor factors are sufficiently correlated, or one predictor represents a linear function of some other predictors. Collinearity is problematic as it leads to a wide array of issues, two most notable issues being (1) problems when separating effects of collinear predictors on the response; and (2) subsequently losing trust in the interpretability of the models and the ability to make inference from the results [24].

To identify potential collinearity issues we used variance inflation factor (VIF) scores. VIF scores indicate the extent to which collinearity is impacting the precision of the least-squares slope coefficients β_i [24]. A higher VIF scores means more of an impact, which is potentially problematic. A common rule of thumb in the Statistics community is that a VIF score, V, where $V < 3$ is generally considered generally non-problematic [12]. In conjunction with VIF scores, we also used raw pairwise correlations between the factors to identify collinearity issues. To address collinearity among a set of predictor variables you can either remove some of the collinear variables, or you can merge them together if that is appropriate on a domain-specific basis.

Through an iterative process using VIF scores and raw pairwise correlations we identified the factors that posed collinearity issues. In our particular dataset, we used a threshold of 0.7 to define correlation indicative of a collinearity issue

The two comment types with the strongest correlation, which was 0.82, were *Solution Discussion* and *Solution Usage*. To determine whether they should be merged or if one should be deleted, we reviewed the code book in the original paper [3]. *Solution Discussion* is predominantly about discussing possible solutions in the solution space, and *Solution Usage* is "usually discussed once a full or partial solution of the issue was released" and includes discussing moving forward with a solution. Since they have such a strong correlation and also both encapsulate discussions about solutions, they were merged together.

Once *Solution Discussion* and *Solution Usage* were merged into the variable *Solution Discussion and Usage*, we observed that all remaining significant pairwise correlations were between *Solution Discussion and Usage* and other variables. *Solution Discussion and Usage* also had a VIF score of 4.13 was is nearly double all the other VIF scores. This suggested *Solution Discussion and Usage* was a major cause of the collinearity issues. Once *Solution Discussion and Usage* was removed, there were no more significant correlations and all the VIF scores were less than three.

Creating Temporal Aggregate Variables

In addition to exploring how total counts of comment types differ across disengaged and engaged contributors, we are also interested in whether there is a difference in the temporal dynamics of comment types. Our data was gathered over eight three-month windows. For each contributor and issue type we have a count for each of the eight periods. Since such long format data would not work well in our logistic regression models and we did not have enough time windows to do dynamic regression or apply full-scale time series modeling, we wanted to create a temporal aggregate variable for each comment type that would encapsulate its change over the eight windows. We created five potential temporal aggregate variables which we evaluated through both holistic visual analysis and statistical tests. Those five potential temporal aggregates

were:

- Slope - We drew a simple linear regression line through the eight time points, and recorded the slope of said line.
- Biggest Drop - We recorded the largest decrease in counts between the consecutive periods.
- Biggest Absolute Change - Similar to *Biggest Drop* except we recorded the largest absolute change instead of the largest decrease.
- Biggest Percent Change - We recorded the largest absolute percent change between periods. I.e., if from P1 to P2 the count dropped 15% and from P2 to P3 the count increased by 40%, then the *Biggest Percent Change* would be 40% assuming there were only three windows in this case.
- Average Change - We recorded the delta difference between the counts for each of the consecutive windows, e.g., δ_1 = P2 - P1. We then calculated the average of these delta values.

Visual Analysis of Temporal Aggregate Variables. We selected a random sample of 10 contributors from our data, created visualizations of the comment counts over time for each comment type, and informally assessed which temporal aggregate variable did the best job overall of capturing the present patterns. From these visualizations, slope appeared to most reliably capture the trends over time. Figure 4.3 shows an example of these visualizations.

Statistical Analysis of Temporal Aggregate Variables. We wanted to explore whether the temporal aggregate variables captured a significant difference across engaged and disengaged contributors since one of these variables would be standing in for the total counts in our temporally-based model.

For each temporal aggregate variable, we created a two-sample t-test for all comments, not each issue type, to see if there was a significant difference between the two groups. A significant difference was observed for all variables but *Biggest Percent Change*.

Selecting the Temporal Aggregate Variable. Based on the holistic visual analysis and statistical tests performed, we determined that slope was the most appropriate temporal aggregate variable to be further used for modeling purposes. It performed well in the statistical tests, visualizations, and, ultimately, by definition we believe it straightforwardly encapsulates the change over time in count for a given comment type best.

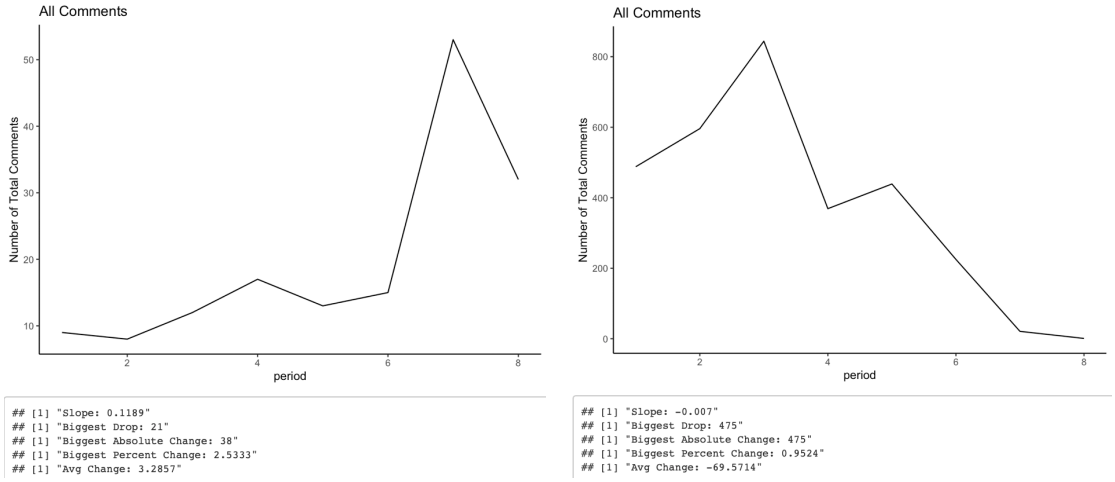


Figure 4.3: Examples of exploratory plots to holistically evaluate the aggregate variables

Creating Operationalized Factors

Based on previous literature, we defined operationalized factors using the issue comment type counts that we believed could be productive when predicting disengagement. The operationalized factors are described below.

- Social Conversation Index - The ratio of their comments that are *Social Conversation* comments.

  Within OSS projects social capital is beneficial to a contributor's long-term engagement on a project [51]. Fang et al. performed a case study OSS contributors where they found that "sustained participation is a natural outcome emerging from successful social interaction between a participation and his or her community" [20].

- Technical Index - The ratio of their comments that are technical comment types, we define technical comment types to be *Expected Behavior*, *Observed Bug Behavior*, *Bug Reproduction*, *Solution Discussion*, *Investigation and Exploration*, *Workarounds*, and *Solution Usage*.

  Developers who do not have a strong social and technical activity intensity, and do not participate in both technical and social activities for long periods of time, coincide with a higher probability of disengagement [14].

- Commitment and Progress Index - The ratio of their comments that are *Contribution and Commitment* or *Task Progress* comments.

  Miller et al. found that how much a contributor works on OSS can be used as a factor to predict their disengagement from the community [45]. These two comment types describe the work a contributor has done or plans to do, so we hypothesize this index will have an analogous relationship with disengagement.

- Action on Issue Index - The ratio of their comments that are *Action on Issue* comments.

  In their case study Lin et al. found that "developers who mainly write code stay longer than developers who mainly work on documentation" [40]. Yu et al. found "shortcomings in project regulation and administration" to be key reasons for "volunteer members dissatisfaction" and that "dissatisfaction was confirmed to be a significant predictor of perceived turnover" [61]. *Action on Issue* comments are comments of an administrative nature that constitute the managing of the issue comments themselves.

- Motivation Index - The ratio of their comments that are *Motivation* comments.

  Within psychological research on job turnover, an employee's motivation is commonly theorized to be an important factor in turnover [38, 39]. We hypothesize that the prevalence of motivation extends to developer disengagement.

4.1.3 Data Exploration

Two-Sample T-Test for Counts and Temporal Slopes

We began by testing whether there was a significant difference in the total count and temporal slopes of the two groups. A two-sample t-test was performed for each comment type using total count and slope.

Two-sampled t-tests compare the mean of two groups and determine whether there is a statistically significant difference between them. Since two-sample t-tests compare the mean of two groups, which can be influenced by outliers, two-sample t-tests can be impacted by outliers.

Because of this, we created box plots for each comment type of both the temporal slope and total count values. We checked to make sure there were no contributors who acted as extreme outliers.

For temporal slope, there were no extreme outliers that would cause concern. For total count, there were some noticeable outliers, so we should be less trusting of the p-values and conclusions coming from those two-sample t-tests. With that said, both the relatively large sample size for each group (at least 230 observations), and the fact the total count t-test results shown in Table 4.1 align with the slope t-test results, increase our confidence in the robustness and generalizability of our overall findings described below in Table 4.1.

Total Count. Each test had a significant p-value and negative t-statistic meaning there is a statistically significant difference in the means of the two groups for all issue comment types, and that the control group (engaged contributors) had the larger mean value for all issue types.

Slope. Two of the tests did not have a significant p-value, the tests for *Solution Discussion* and *Workarounds*. However, all the tests that had a significant p-value also had a negative t-statistic and the control group (engaged contributors) had a larger mean value.

	Total Count t	Total Count p-value	Slope t	Slope p-value
Expected Behavior	-4.2	3.267e-05 ***	-3.0	0.003444 **
Motivation	-3.8	0.0001972 ***	-3.2	0.001714 **
Observed Bug Behavior	-3.5	0.0006078 ***	-3.2	0.001406 **
Bug Reproduction	-4.7	4.452e-06 ***	-3.8	0.0001901 ***
Investigation and Exploration	-3.6	0.0003358 ***	-2.2	0.02694 *
Solution Discussion	-4.6	4.937e-06 ***	-1.8	0.06678 .
Contribution and Commitment	-2.9	0.003554 **	-3.1	0.001971 **
Task Progress	-4.8	1.834e-06 ***	-3.5	0.0005083 ***
Potential New Issues and Requests	-4.6	4.735e-06 ***	-3.3	0.001091 **
Solution Usage	-4.3	2.56e-05 ***	-6.2	1.212e-09 ***
Workarounds	-4.0	9.059e-05 ***	-1.0	0.3759
Action on Issue	-3.9	9.965e-05 ***	-5.1	4.3e-07 ***
Social Conversation	-3.6	0.0003081 ***	-3.6	0.0003654 ***
***p<0.001, **p<0.01, *p<0.05, .p<0.1				

Table 4.1: Results from t-tests for each issue comment type across the two groups using total count and temporal slope metrics

Interaction Models

As a sanity check ✔, for each comment type we created a linear regression model that predicted the total count for the comment type using only disengagement, time period, and the interaction between those two variables.

Linear regression is generally appropriate for situations where the response variable is quantitative and the explanatory variables are either quantitative or categorical [33]. The linear regression modeling technique works by assuming a linear relationship between response and explanatory variables, subsequently estimating the linear effects by minimizing the difference between the actual data points and predicted data points - in our case, using the common least squares approach.

Addressing Interaction Effects. Linear models like linear regression are based on the assumption that the assumed change in response based on the change in one explanatory variable is independent of any change in the other explanatory variables. However there are many situations where from either domain knowledge or data exploration we know this is not true. We call this effect between two response variables an *interaction effect*. In circumstances like these, we add an *interaction term* to our model which accounts for the relationship between two explanatory variables as it

relates to the response variable [33].

An an example, consider the model predicting the total count of *Expected Behavior* comments using *Disengaged* and *Period*, which represent whether the contributor has disengaged and the time period, respectively. Let X_1 and X_2 be the values for *Disengaged* and *Period* respectively, and let β_0, β_1 and β_2 be the coefficients for intercept, *Disengaged* and *Period*, respectively, with Y denoting the response (*Expected Behavior* total count) and ϵ - error terms. This gives us the model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$$

We can see that in this model, if we increase X_1 by a unit, holding X_2 constant, then we expect Y to increase by β_1 units, regardless of the specific value X_2 is held constant at. To make the X_1 effect on Y dependent on the value of X_2, we can introduce a new predictor - $X_1 X_2$ - that represents the interaction between these two variables as shown below.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \epsilon$$

Now the change in X_1 is non-constant and changes in X_2 will change the relationship between X_1 and Y.

Since X_2 is a binary variable where 0 is engaged and 1 means disengaged, we can see that once the interaction term is added, the regression lines for disengaged and engaged contributors will have both different slopes and intercepts. When we compare a plot these two lines, if they are parallel, that is indicative of there not being a interaction effect. However, if there is a difference in slopes than that signifies that there is an interaction effect.

Table 4.2 shows the interaction coefficients from the linear regression models.

Model Diagnostics. We performed standard diagnostics on these models including visualizations to ensure the normality, linearity, and constant variance assumptions were being met. We used a normal-QQ plot to check for normality, and a residuals versus fitted plot to check for linearity and constant variance. The diagnos-

Comment Type	Disengaged:Period Coefficient	P-value
Solution Discussion	-4.46	0.000133 ***
Contribution and Commitment	-0.45	0.00704 **
Task Progress	-0.42	4.32e-06 ***
Action on Issue	-0.29	0.00665 **
Bug Reproduction	-0.29	0.00132 **
Solution Usage	-0.18	0.000848 ***
Motivation	-0.14	0.000383 ***
Expected Behavior	-0.07	0.000933 ***
Potential New Issues and Requests	-0.06	0.00114 **
Observed Bug Behavior	-0.04	0.00317 **
Investigation and Exploration	-0.03	0.0148 *
Social Conversation	-1.25	0.0292 *
Workarounds	-0.02	0.0402 *
***p<0.001, **p<0.01, *p<0.05, .p<0.1		

Table 4.2: Interaction coefficient estimates from linear regression models predicting the total count for each issue comment type

tic plots revealed that the assumptions were not in fact being met. Through further investigation it was revealed that there were issues with zero-inflation. Zero-inflation means there are more zero counts in the data than is expected under classic normal distribution, therefore violating the modeling assumptions. Figure 4.4 shows plots of the frequency distributions for two of the comment types which emphasize the overwhelming prevalence of zero counts, and subsequent need for modeling adjustments.

Addressing Zero-Inflation. To address the zero-inflation, we introduced the Zero-Inflated Poisson (ZIP) model. ZIP models account for zero-inflation by using two different modeling components. Imagine that out of all of the data cases, there are two "latent" classes of observations: those that could have a non-zero response count, and those who would have a zero response count no matter what ("0-class"). In our specific case, for example, something could have happened to a person healthwise, or they were placed in a location from which they could not contribute, hence resulting into 0 comments no matter what, belonging to that 0-class. ZIP models work by having two distinct modeling components for these two observation classes, with a Poisson regression model used for observations allowed to have non-zero count response values (e.g. allowed to post any non-negative number of comments), while a

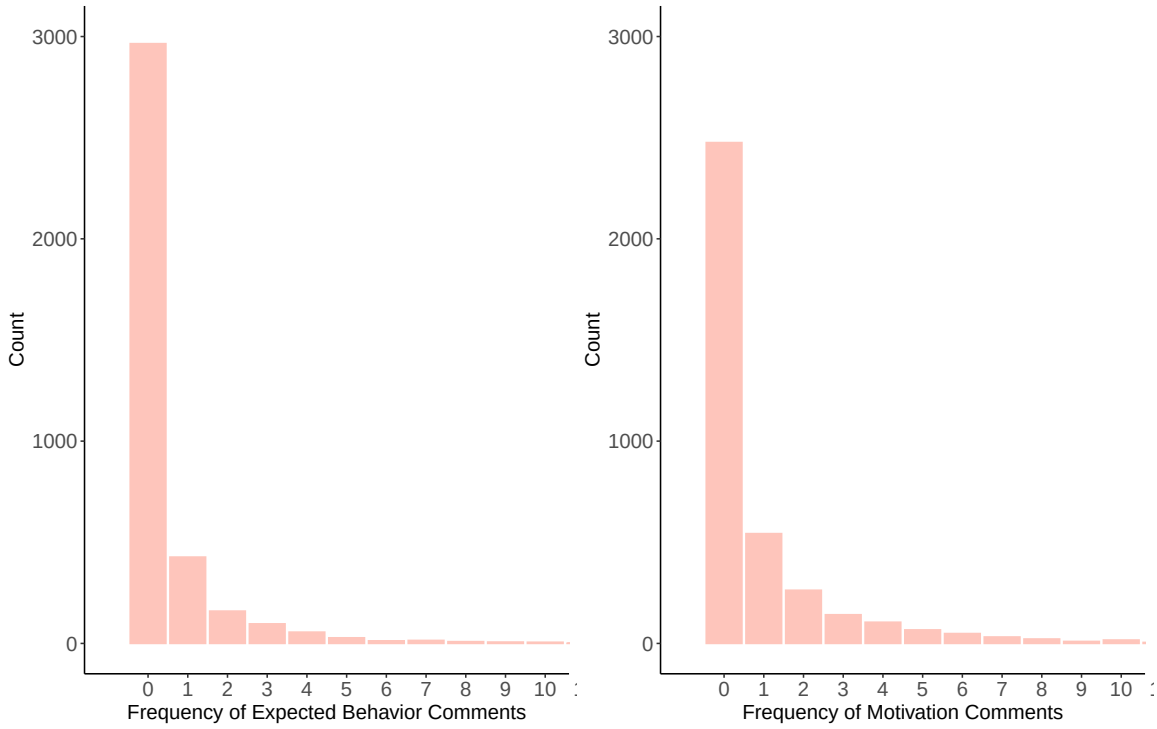


Figure 4.4: Example of the frequency distribution found among the comment types highlighting the zero-inflation issue.

binary regression model is utilized to determine the chance of whether an observation belongs to the "0-class" (e.g. in a situation where they physically can not write a comment no matter what).

The coefficient estimates for the interaction effect in each of the ZIP models are listed in Table 4.3. For all of the models, the interaction coefficient was negative and statistically significant. We would expect that because the activity of disengaged users (corresponding to 1-value of the disengagement dummy variable) would decrease at a faster rate over time than for engaged users, hence having a more negative slope.

From Table 4.3 we can see that the comment types that have the most practical significance in terms of magnitude of the interaction coefficient are *Workarounds*, *Investigation and Exploration*, and *Task Progress*. The comment types with the most statistically significant interaction coefficients in terms of p-value are *Task Progress*, *Solution Discussion*, *Social Conversation*, and *Contribution and Commitment*. It should be noted that *Task Progress* has one of the most significant interaction coef-

ficient in terms of both statistical and practical significance.

Comment Type	Disengaged:Period Coefficient	P-Value
Expected Behavior	-0.094953	0.00217 **
Motivation	-0.1232434	4.1e-12 ***
Observed Bug Behavior	-0.10818	0.0385 *
Bug Reproduction	-0.0785979	2.52e-13 ***
Investigation and Exploration	-0.17932	0.0005 ***
Solution Discussion	-0.134219	<2e-16 ***
Contribution and Commitment	-0.095610	<2e-16 ***
Task Progress	-0.14438	<2e-16 ***
Potential New Issues and Requests	-0.104761	0.0016 **
Solution Usage	-0.064000	4.94e-06 ***
Workarounds	-0.19551	0.00956 **
Action on Issue	-0.090708	7.26e-14 ***
Social Conversation	-0.114672	<2e-16 ***
***p<0.001, **p<0.01, *p<0.05, .p<0.1		

Table 4.3: Results from ZIP model for each issue comment type across the two groups using total count and slope metrics

4.1.4 Model Building

To explore the relationship between issue comment types and disengagement we created two logistic regression models, each with a different representation of the issue comment types which we will refer to as the *issue comment variables*.

The *issue comment variables* for the first model are the total counts, i.e., the total number of comments of a given issue comment type made by a contributor over all eight periods. We will refer to this model as the Static-Model, as it doesn't account for the temporal dynamics of the comment counts.

The second model uses the temporal aggregate variable -slope- as the *issue comment variables* to represent the progression for each issue comment type over time, and will subsequently be referred to as the Non-Static-Model.

We followed the same procedure when building both the Static-Model and Non-Static-Model which we describe below.

- *Step 1.* Iteratively Build Logistic Regression Model. We used a logistic regression model which is a form of generalized linear model that is appropriate for

situations like ours were we have a binary factor as our response variable [24]. Unlike linear regression, which provides a prediction of the quantitative response value (e.g., comment count), logistic regression predicts the log-odds or logit that the response variable belongs to a specific class (e.g., engaged or disengaged), using the maximum likelihood method to estimate linear effects of explanatory variables [24].

We started by building a model using only the *issue comment variables* as the explanatory variables, then we checked for and addressed collinearity issue by using the process described in the next step. Having addressed the collinearity concerns, we added the operationalized variables, checked for collinearity once again, then added the control variables, and checked for collinearity one last time. At the end we had a full model with all the remaining factors after collinearity had been addressed.

- *Step 2.* Collinearity Check. To check for and address issues of collinearity we used both VIF scores and significant correlation between variables. We defined significant correlation between variables X and Y to be $cor(X, Y) \geq |0.6|$. We removed one variable at a time until there were no variables with a significant correlation and all the VIF scores were less than 5 which is a common threshold in the statistics community.

- *Step 3.* Variable Selection. We then used backwards stepwise regression which removes one variable at a time based on a given criterion until no more variables can be removed without a strong deterioration to the quality of model fit [24]. We used the Akaike Information Criterion (AIC), which considers two main components in its calculation (1) the number of parameters or explanatory variables in the model; and (2) the quality of fit for the model, measured using the maximum likelihood approach [1].

- *Step 4.* Model Diagnostics. We used the DHARMa[1] R package for the logistic regression diagnostics.

[1]https://cran.r-project.org/web/packages/DHARMa/vignettes/DHARMa.html

The DHARMa uses simulation-based techniques to create simulated residuals for logistic regression as well as other generalized linear models. These simulated residuals are "standardized to values between 0 and 1 and can be interpreted as intuitively as residuals from a linear regression." What this means is that, first, DHARMa generates new synthetic data sets by simulating the fitted model, then the cumulative distribution of the simulated values is calculated, and finally the quantile for the residual value is calculated which in turn corresponds with the observed value. This is why the simulated residuals are always non-negative and between 0-1. Once we have generated the simulated residuals, next we can create the DHARMa version of the same plots we used to diagnose our linear regression models in Section 4.1.3.

The first plot we generate is a QQ plot of the residuals so we can check whether they align with the expected distribution. The axis labels 'Expected' represents what the residual values are expected to be if modeling assumptions were satisfied, and the 'Observed' label represents the actual simulated residuals. At the top of the plot we also see the results of a Kolomogorov-Smirnov Test (KS Test) which is used to evaluate whether there is a significant difference between two samples with the purpose of determining the uniformity of a sample, in our case the simulated residuals [42]. The null hypothesis is that the sample in question is uniformly distributed between 0 and 1 and this helps us test for the uniformity of our residuals. We then create a residual versus fitted plot to check for the standard linearity. The dashed black lines represent the theoretical regressions for the 0.25, 0.5, and 0.75 quantiles.

- *Step 5. Evaluating Performance and Model Validation* To evaluate the model, we perform ANOVA tests which evaluates the overall model significance. We also review the coefficient estimates for the predictors and check the accuracy of the models. When considering the accuracy of our model, we first checked the accuracy on the training data. In addition we used Leave One Out Cross Validation (LOOCV) like we did to evaluate the Updated-Model in Phase 1.

- *Step 6. Data Reflection* We visualized a Principal Component Analysis (PCA) technique using the subset of variables present in the final reduced model to determine if we could visually identify any of the results in the model summary. We also checked the accuracy for the model and compared it to a baseline model which always predicts a contributor is engaged. The baseline model predicts that contributors are engaged because engaged contributors are the majority class of our training data and so this model has the highest performance in terms of accuracy out the simple models which always predict one class.

  PCA is an unsupervised learning technique, where the response variable does not have to be supervised (although we mark the engaged disengaged observations on the plot, we do it after the PCA has already been performed purely on predictor variables – it never needed that response variable knowledge). PCA uses dimensionality reduction to understand the relationships between variables. Suppose we have a logistic regression model with two quantitative explanatory variables. If we want to visualize the relationship between them, we can do a scatter plot. However, if we have k explanatory variables where $k \geq 4$ we can no longer make a single visualization of the relationship between all the variables. This is where PCA modeling comes into play.

  PCA reduces down the k-dimensional space while capturing as much information as possible. Assume there are n observations and k explanatory variables, $X_1, X_2, ..., X_k$. Since there are k dimensions, we can think of each of the n observations exists in a k-dimensional space. If we compare the space for two arbitrary observations, there will likely be differences among each of the dimensions, however, PCA attempts to identify a subset of dimensions that have the most variance and therefore capture the most information. The dimensions in this subset are called the *principal components* and they are a "normalized linear combination of the features" with the most variance [33].

	Estimate	Std. Error	Z Value	P-Value
(Intercept)	0.198308	0.327880	0.605	0.54530
Workarounds	-0.113085	0.060258	-1.877	0.06056 .
Action on Issue	-0.018804	0.006115	-3.075	0.00211 **
Social Conversation Index	1.184558	0.639586	1.852	0.06402 .
Percent95	-0.007435	0.004559	-1.631	0.10290
PercentSupport	0.028778	0.009460	3.042	0.00235 **
Full Model Significance				3.514e-08 ***
***p<0.001, **p<0.01, *p<0.05, .p<0.1				

Table 4.4: Summary table from reduced Static-Model

4.2 Results

4.2.1 Static-Model

Table 4.4 shows the results of fitting the Static-Model. These results can be trusted because the Static-Model passed the DHARMa residual diagnostic checks as shown in Figure 4.5. Using ANOVA to test for full model significance, we found that the full model was significant with a p-value of 3.514e-08.

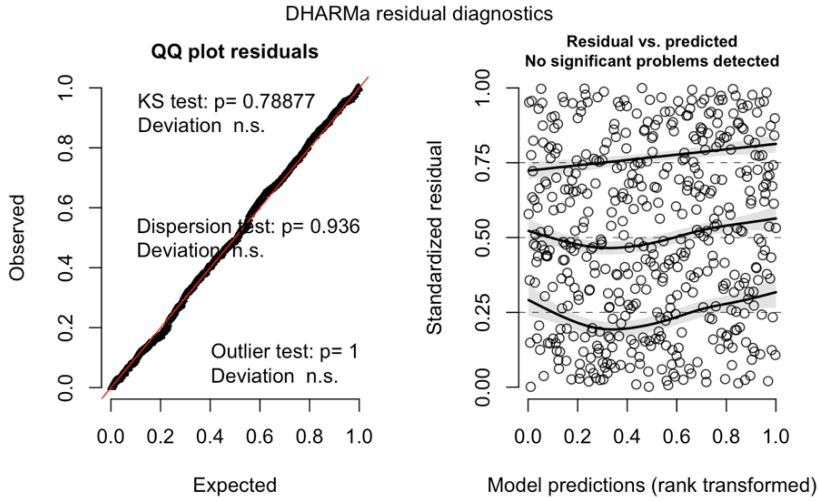


Figure 4.5: DHARMa Output for Static-Model

The Static-Model only contained two *issue comment variables*, *Workarounds* and *Action on Issue*, with the latter (*Action on Issue*) being the only one with a statisti-

cally significant coefficient. We can interpret this coefficient as telling us that, holding several other factors constant (workarounds, social conversation, percent of commits posted between 7am & 7pm, percent of support work), higher counts of *Action on Issue* comments lead to lower probability of disengagement.

Social Conversation Index was the only operationalized factor present. From the control variables, both *Percent95* and *PercentSupport* were present. The Static-Model results are shown in Table 4.4.

PercentSupport also had a statistically significant coefficient estimate in this model. We can interpret the *PercentSupport* coefficient estimate of 0.029 as saying for every percentage point increase in *PercentSupport*, we expect the odds of disengaging to multiply by $e^{0.029} = 1.03$, or increase by 3%, when controlling for the other variables. This tells us that doing more support work can lead to higher odds of disengagement. These findings align with previous work from Miller et al. who also studied how *PercentSupport* affected odds of disengagement, in their work they found that having high *PercentSupport* increased a the risk of disengagement by a factor of 1.43 [45]. However it is important to note this coefficient did not have a significant p-value in their model so we do not have high confidence, but instead consider it an informal anecdote.

4.2.2 Non-Static-Model

Table 4.5 shows the results of fitting the Non-Static-Model. These results are trustworthy because the model passes the residual diagnostic checks as can be seen in Figure 4.6. We can observe from the QQ plot (left-hand side of Figure 4.6) that the simulated residuals align with the expected distribution, and since the KS test has an insignificant of p-value of 0.8 we fail to reject the null hypothesis that the residuals are uniform. From the residuals-versus-fitted (right-hand side of Figure 4.6) plot we observe that the quantiles align as expected, without any significant outliers (which, if present, would've been indicated with red color) Using ANOVA to test full model significance, we found that the overall model was significant and had a p-value of 1.369e-09.

	Estimate	Std. Error	Z Value	P-Value
(Intercept)	-0.42906	0.17338	-2.475	0.013336 *
Observed Bug Behavior Slope	-0.15671	0.07778	-2.015	0.043913 *
Bug Reproduction Slope	-0.16650	0.08556	-1.946	0.051654 .
Potential New Issues and Requests Slope	-0.12499	0.07910	-1.580	0.114061
Action on Issue Slope	-0.37660	0.10314	-3.651	0.000261 ***
Social Conversation Slope	-0.24001	0.11129	-2.157	0.031043 *
Social Conversation Index	1.30091	0.65979	1.972	0.048643 *
Full Model Significance				1.369e-09 ***
***p<0.001, **p<0.01, *p<0.05, .p<0.1				

Table 4.5: Results from reduced Non-Static-Model

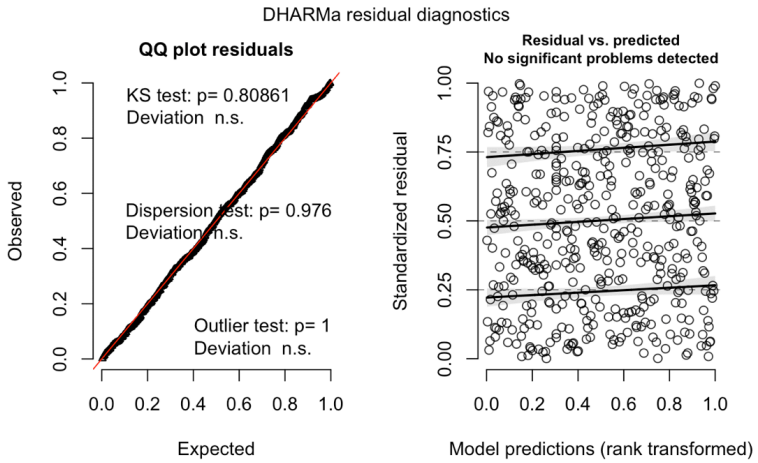


Figure 4.6: DHARMa Output for Non-Static-Model

The *issue comment variables* that are present in the reduced Non-Static-Model are *Social Conversation*, *Action on Issue*, and *Observed Bug Behavior* all of which have a significant coefficient. *Bug Reproduction* and *Potential New Issues and Requests* were also present in the reduced model.

Since the coefficients for *Social Conversation*, *Action on Issue*, and *Observed Bug Behavior* are less than 0, we can say the faster these types of comments grow over time, holding other factors constant, the lower the chance of contributor disengaging.

Action on Issue in particular is worthy of note because it was the only comment type that was significant in both the Static-Model and Non-Static-Model. We can interpret the *Action on Issue* coefficient in the Non-Static-Model as saying for every

additional *Action on Issue* comment per time period, we expect the odds of disengaging to multiply by $e^{-0.38} = 0.68$. In other words, as a contributor does more *Action on Issue comments*, the probability of disengagement decreases by 32%, holding other variables constant.

4.2.3 Comparing Model Performances

The Static-Model has an in-sample accuracy of 0.62 and the Non-Static-Model has an in-sample accuracy of 0.65. Both the models significantly out performed the baseline model in terms of accuracy, which had an in-sample accuracy of 0.51. The Non-Static-Model edges out the Static-Model in terms of performance based on in-sample accuracy. We conjecture that this is in large part due to the Non-Static-Model accounting for temporal dynamics of the issue comments, rather than just their cumulative totals.

Static-Model			Non-Static-Model		
Pred \ True	0	1	Pred \ True	0	1
0	143 (0.59)	81 (0.35)	0	164 (0.67)	89 (0.39)
1	100 (0.41)	150 (0.65)	1	79 (0.33)	142 (0.61)

Table 4.6: Confusion matrix of in-sample performance of Static-Model and Non-Static-Model (0 = engaged, 1 = disengaged). Specifically, each cell contains observation counts corresponding to that combination of true and predicted value, followed in parentheses by classification rates for observations of the corresponding true class.

Table 4.6 shows the confusion matrix of the in-sample performance for both the Static-Model and Non-Static-Model. We can see that the Static-Model had a higher false positive rate (0.41). A false positive in this circumstance is misclassifying a contributor who had not disengaged as having disengaged. Additionally, we we see the Non-Static-Model has a higher false negative rate (0.39). A false negative in this circumstance means misclassifying a contributor who disengaged as not disengaging.

Given our use case of identifying disengaged contributors so subsequent interventions can potentially be held to prevent disengagement, we care more about catching

all contributors who are disengaging meaning we would rather have a higher false positive rate and lower false negative rate, giving a slight edge to Static-Model in that regard. With that said, performances of two models in identifying disengaged users are very comparable.

Lastly, we review the out-of-sample performance using LOOCV. The Static and Non-Static models had an out-of-sample accuracy of 0.61 and 0.62 respectively. While the difference in performance is smaller, the Non-Static-Model still outperformed the Static-Model and had a slightly higher overall out-of-sample prediction accuracy.

4.2.4 Data Reflection

Even though the main goal of our analysis was of interpretational nature - figuring out the most impactful factors potentially leading to contributor disengagement - predictive performance should not be completely ignored when verifying the legitimacy of our results. With none of the models achieving higher than 65% overall accuracy, or lower than 35% false positive/negative rates, we are interested in further diagnosing why that might be the case. Specifically, we will conduct a postmortem analysis to check how separable the variables are in the feature sets selected by the models.

Figures 4.7 and 4.8 show visualizations from PCA conducted on Static and Non-Static models, respectively. For the most part, there is hardly any visible separation between classes in both models, giving an intuitive explanation to the low classification accuracy values. With that said, Non-Static model appears to have a noticeable majority of engaged contributors for higher values of *Action on Issue, Social Conversation, Potential New Issues and Requests, Bug Reproduction,* and *Observed Bug Behavior* **temporal slope** predictors (right-hand side of Figure 4.8), while most disengaged contributors are on the lower end of the spectrum for these predictors (left-hand side of Figure 4.8). Static model's cumulative features, such as *Action On Issue* or *WorkAround* comment **totals**, do not appear to separate the two classes as well (see 4.7), likely leading to lower accuracy of 62% as opposed to Non-Static model's 65%.

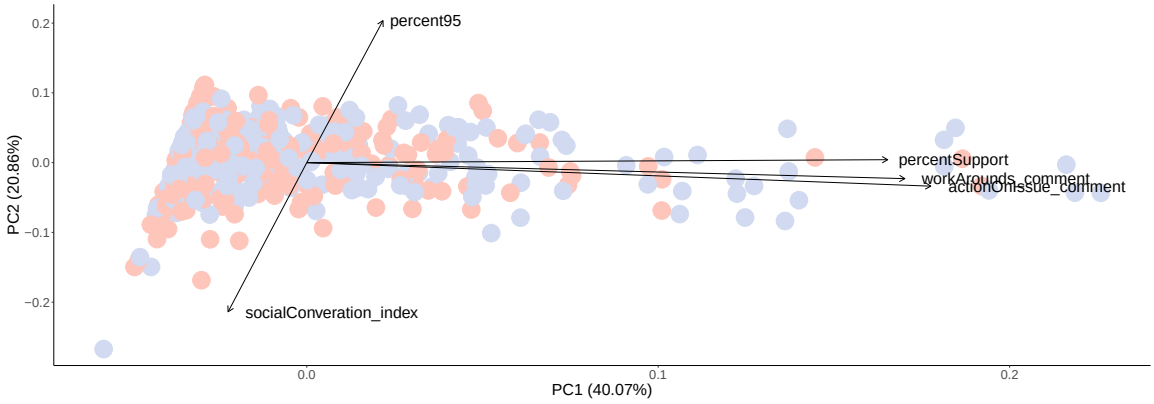


Figure 4.7: PCA for variables in final reduced Static-Model
Pink = Disengaged, Blue = Engaged

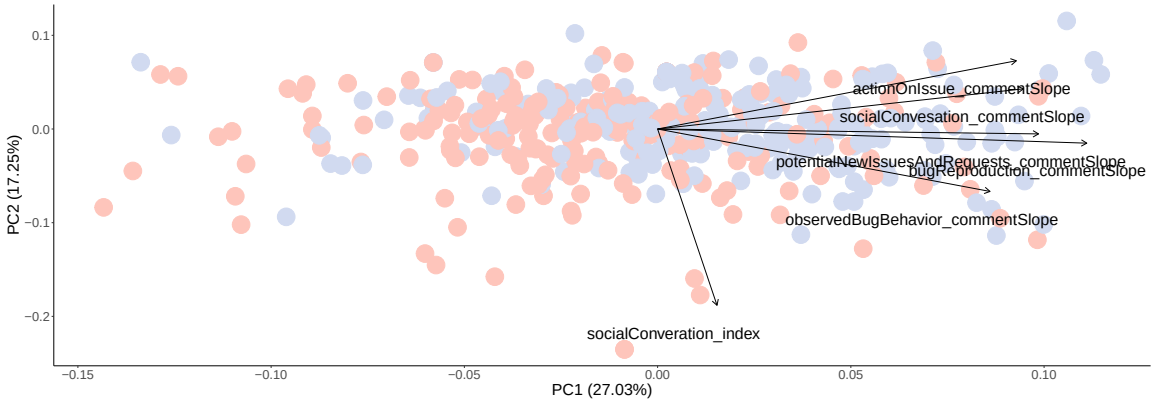


Figure 4.8: PCA for variables in final reduced Non-Static-Model.
Pink = Disengaged, Blue = Engaged

Chapter 5

Conclusion and Future Work

5.1 Threats to Validity

Here we discuss some of the threats to validity of our paper:

In Section 3.2.2 we discuss how the Updated-Model had an overall F1-Score of 0.3, which was 0.12 lower than the Previous-Model's score of 0.42. We used the Updated-Model to classify the comments in Phase 2, and since the model does not have perfect performance some of the issue comments may have been misclassified. This misclassification of issue comments could have a subsequent impact on the findings of Phase 2 so caution should be taken when discussing the findings.

The replication training data consisted of 15 issue comment threads originating from four popular AI libraries on GitHub. Since the classifier was trained on these comments, which were exclusively from the Software Engineering sub-domain of AI libraries, there are questions on how well the classifier performs on all the issue comments in Phase 2 since they are the issue comments from a random sample of contributors who likely contribute to a wide array of open source projects hosted on GitHub.

Although the definition of core contributor disengagement used in this paper was a more dynamic version of the definition used in previous research [45], it still may not capture the disengagement of all core contributors, especially those who disengage more gradually over a longer period of time.

In addition, the core contributors we identified were all from a specific point in time, so caution must be taken when generalizing the findings to a wider group of contributors.

The factors we operationalized, as with any other model, do not capture the full concept we are trying to encapsulate. Therefore caution must be taken when generalizing our findings beyond the particular operationalizations from this paper.

When selecting slope as our temporal aggregate variable, we used an informal holistic approach to make the decision leading to a certain level of subjectivity in that decision.

While we used rigorous diagnostics on most of our models, we did not perform thorough model diagnostics on the Zero-Inflated Poisson regression model to check whether the data ends up fully satisfying distributional assumptions. This is in large part due to Zero-Inflated model diagnostics being a less explored area of statistics.

5.2 Conclusions and Future Work

Understanding open source disengagement is important because contributor disengagement, particularly core contributor disengagement, has a significant impact on the survival of a project. We care about the survival of a single project because open source projects exist in an ecosystem with a network of interdependencies so the failure of a single project can have cascading impacts on the rest of the ecosystem. The survival of open source ecosystems is critical because lots of modern software from Instagram to Google rely on open source.

This thesis represents a small piece of the puzzle in understanding open source sustainability. We examined the role that communication plays in core contributor disengagement. Specifically, we have investigated how the type of issue comments written by core contributors relate to disengagement.

By learning more about developer disengagement we hope to understand the underlying causes so eventually tools and other interventions can be made to intervene before disengagement occurs. This should ultimately help improve the sustainability

of the open source ecosystems as a whole.

In Phase 1 we built the Updated-Model classifier inspired by the Previous-Model architecture of Arya *et al.* with some modifications. First, we addressed class imbalance in the training data differently, opting for SMOTE upsampling rather than adjusting class weights. Second, we modified the classifier architecture to include an LSTM layer and created the sentence-level representations, which act as input to the classification layer, differently. Ultimately, our overall performance was slightly worse than that of the Previous-Model.

In Phase 2 we used the classifier from Phase 1 to categorize the issue comments of core contributors over a two year period that either ended in disengagement or continued engagement.

We first examined whether there was a meaningful difference in the frequency and pattern of issue comments written by the groups. To accomplish this we performed two-sample t-tests to determine whether there was a significant difference in average number of issue comments across the groups for each issue comment type, which we confirmed there was. Next we we created interaction models predicting the total number of comments of each type over time using whether a contributor had disengaged or not and the period (P1-P8). We found that the issue comments of disengaged contributors decreased at a faster pace over time compared to the comments of contributors who did not disengage.

Next we studied how the comment dynamics of different issue types relate to the likelihood of contributor disengagement. We created two logistic regression models to classify disengagement using two different representations for the issue comment types as the explanatory variables, along with the control and operationalized variables from previous literature which were the same in both models. In the first model, the Static-Model, issue comment types were represented by the total count of that type over the eight time windows. In the second model, the Non-Static-Model issue comment types were represented by the slope of the count over the eight time periods.

Both our logistic regression models confirmed that developers who contribute more *Action on Issue* comments have lower odds of disengaging. This is interesting because

Action on Issue comments are discussions about the management of the issue comments itself, which suggests a certain amount of responsibility or 'ownership' of an issue thread. The Non-Static-Model which accounted for the temporal dynamic of the comments slightly outperformed the Static-Model.

In summation, we learned that in the time leading up to and including disengagement, (1) there is a significant difference in the total number of issue comments across the two groups for all kinds of issue comments measured, (2) the comments of disengaged contributors decrease at a faster pace; (3) contributing more *Action on Issue* comments leads to lower probability of disengagement.

Future avenues of work could explore whether the same relationship holds when core contributors and their disengagement are defined differently as has been done in recent work [11, 8]. In this thesis, we considered the issue comments written by contributors over a two year period that either ended in disengagement or continued contributions. Future work could take a more longitudinal look at the kinds of issue comments written by contributors. They could also focus more closely on a smaller number of comment types that may relate more closely to disengagement like *Action on Issue* and *Motivation*.

Additionally, future work would strive to improve the automated issue comment classifier architecture by experimenting with alternative methods for addressing class imbalance and representing the textual data as numeric inputs. In this paper, we used the training data from the replication package by Arya *et al.* which consisted of 15 issue threads from popular AI libraries on GitHub. Future work could also consider expanding the training data to include issue threads from a wider spectrum of open source projects and using other types of text from GitHub like pull request comments.

Bibliography

[1] Hirotugu Akaike. A new look at the statistical model identification. IEEE transactions on automatic control, 19(6):716–723, 1974.

[2] Khaled Albusays, Pernille Bjorn, Laura Dabbish, Denae Ford, Emerson Murphy-Hill, Alexander Serebrenik, and Margaret-Anne Storey. The diversity crisis in software development. IEEE Software, 38(2):19–25, 2021.

[3] Deeksha Arya, Wenting Wang, Jin LC Guo, and Jinghui Cheng. Analysis and detection of information types of open source software issue discussions. In 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), pages 454–464. IEEE, 2019.

[4] Guilherme Avelino, Leonardo Passos, Andre Hora, and Marco Tulio Valente. A novel approach for estimating truck factors. In 2016 IEEE 24th International Conference on Program Comprehension (ICPC), pages 1–10. IEEE, 2016.

[5] Frederick P Brooks Jr. The mythical man-month: essays on software engineering. Pearson Education, 1995.

[6] Stefan Büttcher, Charles LA Clarke, and Gordon V Cormack. Information retrieval: Implementing and evaluating search engines. Mit Press, 2016.

[7] Fabio Calefato, Giuseppe Iaffaldano, Filippo Lanubile, and Bogdan Vasilescu. On developers' personality in large-scale distributed projects: the case of the apache ecosystem. In 2018 IEEE/ACM 13th International Conference on Global Software Engineering (ICGSE), pages 87–96. IEEE, 2018.

[8] Edna Dias Canedo, Rodrigo Bonifácio, Márcio Vinicius Okimoto, Alexander Serebrenik, Gustavo Pinto, and Eduardo Monteiro. Work practices and perceptions from women core developers in oss communities. In Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pages 1–11, 2020.

[9] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: synthetic minority over-sampling technique. Journal of artificial intelligence research, 16:321–357, 2002.

[10] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, pages 186–196, 2017.

[11] Jailton Coelho, Marco Tulio Valente, Luciana L Silva, and André Hora. Why we engage in floss: Answers from core developers. In Proceedings of the 11th International Workshop on Cooperative and Human Aspects of Software Engineering, pages 114–121, 2018.

[12] Jacob Cohen, Patricia Cohen, Stephen G West, and Leona S Aiken. Applied multiple regression/correlation analysis for the behavioral sciences. Routledge, 2013.

[13] Jorge Colazo and Yulin Fang. Impact of license choice on open source software development activity. Journal of the American Society for Information Science and Technology, 60(5):997–1011, 2009.

[14] Eleni Constantinou and Tom Mens. An empirical comparison of developer retention in the rubygems and npm software ecosystems. Innovations in Systems and Software Engineering, 13(2-3):101–115, 2017.

[15] Kevin Crowston, Hala Annabi, James Howison, and Chengetai Masango. Effective work practices for software engineering: free/libre open source software

development. In Proceedings of the 2004 ACM workshop on Interdisciplinary software engineering research, pages 18–26, 2004.

[16] Giuseppe Destefanis, Marco Ortu, Steve Counsell, Stephen Swift, Michele Marchesi, and Roberto Tonelli. Software development: do good manners matter? PeerJ Computer Science, 2:e73, 2016.

[17] Carolyn D Egelman, Emerson Murphy-Hill, Elizabeth Kammer, Margaret Morrow Hodges, Collin Green, Ciera Jaspan, and James Lin. Predicting developers' negative feelings about code review. In 2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE), pages 174–185. IEEE, 2020.

[18] Nadia Eghbal. Roads and bridges: The unseen labor behind our digital infrastructure. Ford Foundation, 2016.

[19] Hongbo Fang, Daniel Klug, Hemank Lamba, James Herbsleb, and Bogdan Vasilescu. Need for tweet: How open source developers talk about their github work on twitter. In Proceedings of the 17th International Conference on Mining Software Repositories, pages 322–326, 2020.

[20] Yulin Fang and Derrick Neufeld. Understanding sustained participation in open source software projects. Journal of Management Information Systems, 25(4):9–50, 2009.

[21] Joseph Feller, Brian Fitzgerald, et al. Understanding open source software development. Addison-Wesley London, 2002.

[22] Denae Ford, Mahnaz Behroozi, Alexander Serebrenik, and Chris Parnin. Beyond the code itself: how programmers really look at pull requests. In 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS), pages 51–60. IEEE, 2019.

[23] Matthieu Foucault, Marc Palyart, Xavier Blanc, Gail C Murphy, and Jean-Rémy Falleri. Impact of developer turnover on quality in open-source soft-

ware. In Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, pages 829–841, 2015.

[24] John Fox. Applied regression analysis and generalized linear models. Sage Publications, 2015.

[25] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. Deep learning, volume 1. MIT press Cambridge, 2016.

[26] Georgios Gousios. The ghtorent dataset and tool suite. In 2013 10th Working Conference on Mining Software Repositories (MSR), pages 233–236. IEEE, 2013.

[27] Georgios Gousios, Margaret-Anne Storey, and Alberto Bacchelli. Work practices and challenges in pull-based development: the contributor's perspective. In 2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE), pages 285–296. IEEE, 2016.

[28] Daniel Graziotin, Fabian Fagerholm, Xiaofeng Wang, and Pekka Abrahamsson. What happens when software developers are (un) happy. Journal of Systems and Software, 140:32–47, 2018.

[29] Christoph Hannebauer and Volker Gruhn. On the relationship between newcomer motivations and contribution barriers in open source projects. In Proceedings of the 13th International Symposium on Open Collaboration, pages 1–10, 2017.

[30] Guido Hertel, Sven Niedner, and Stefanie Herrmann. Motivation of software developers in open source projects: an internet-based survey of contributors to the linux kernel. Research policy, 32(7):1159–1177, 2003.

[31] Yu Huang, Denae Ford, and Thomas Zimmermann. Leaving my fingerprints: Motivations and challenges of contributing to oss for social good. In 2021 IEEE/ACM 43th International Conference on Software Engineering (ICSE). IEEE, 2021.

[32] Daniel Izquierdo-Cortazar, Gregorio Robles, Felipe Ortega, and Jesus M Gonzalez-Barahona. Using software archaeology to measure knowledge loss in software projects due to developer turnover. In 2009 42nd Hawaii International Conference on System Sciences, pages 1–10. IEEE, 2009.

[33] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. An introduction to statistical learning, volume 112. Springer, 2013.

[34] Nathalie Japkowicz and Shaju Stephen. The class imbalance problem: A systematic study. Intelligent data analysis, 6(5):429–449, 2002.

[35] Mitchell Joblin, Sven Apel, Claus Hunsen, and Wolfgang Mauerer. Classifying developers into core and peripheral: An empirical study on count and network metrics. In 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), pages 164–174. IEEE, 2017.

[36] Jymit Khondhu, Andrea Capiluppi, and Klaas-Jan Stol. Is it all lost? a study of inactive open source projects. In IFIP international conference on open source systems, pages 61–79. Springer, 2013.

[37] Karim R Lakhani and Robert G Wolf. Why hackers do what they do: Understanding motivation and effort in free/open source software projects. 2003.

[38] Thomas W Lee and Terence R Mitchell. The unfolding effects of organizational commitment and anticipated job satisfaction on voluntary employee turnover. Motivation and Emotion, 15(1):99–121, 1991.

[39] Thomas W Lee and Terence R Mitchell. An alternative approach: The unfolding model of voluntary employee turnover. Academy of management review, 19(1):51–89, 1994.

[40] Bin Lin, Gregorio Robles, and Alexander Serebrenik. Developer turnover in global, industrial open source projects: Insights from applying survival analysis. In 2017 IEEE 12th International Conference on Global Software Engineering (ICGSE), pages 66–75. IEEE, 2017.

[41] Bin Lin, Fiorella Zampetti, Gabriele Bavota, Massimiliano Di Penta, Michele Lanza, and Rocco Oliveto. Sentiment analysis for software engineering: How far can we go? In Proceedings of the 40th International Conference on Software Engineering, pages 94–104, 2018.

[42] Frank J Massey Jr. The kolmogorov-smirnov test for goodness of fit. Journal of the American statistical Association, 46(253):68–78, 1951.

[43] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781, 2013.

[44] Courtney Miller, Paige Rodeghero, Margaret-Anne Storey, Denae Ford, and Thomas Zimmermann. "how was your weekend?" software development teams working from home during covid-19. In 2021 IEEE/ACM 43th International Conference on Software Engineering (ICSE). IEEE, 2021.

[45] Courtney Miller, David Gray Widder, Christian Kästner, and Bogdan Vasilescu. Why do people give up flossing? a study of contributor disengagement in open source. In IFIP International Conference on Open Source Systems, pages 116–129. Springer, 2019.

[46] Andreas C Müller and Sarah Guido. Introduction to machine learning with Python: a guide for data scientists. " O'Reilly Media, Inc.", 2016.

[47] Kumiyo Nakakoji, Yasuhiro Yamamoto, Yoshiyuki Nishinaka, Kouichi Kishida, and Yunwen Ye. Evolution patterns of open-source software systems and communities. In Proceedings of the international workshop on Principles of software evolution, pages 76–85, 2002.

[48] Cassandra Overney, Jens Meinicke, Christian Kästner, and Bogdan Vasilescu. How to not get rich: An empirical study of donations in open source. In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pages 1209–1221, 2020.

[49] Gustavo Pinto, Clarice Ferreira, Cleice Souza, Igor Steinmacher, and Paulo Meirelles. Training software engineers using open-source software: the students' perspective. In 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET), pages 147–157. IEEE, 2019.

[50] Huilian Sophie Qiu, Yucen Lily Li, Susmita Padala, Anita Sarma, and Bogdan Vasilescu. The signals that potential contributors look for when choosing open-source projects. Proceedings of the ACM on Human-Computer Interaction, 3(CSCW):1–29, 2019.

[51] Huilian Sophie Qiu, Alexander Nolte, Anita Brown, Alexander Serebrenik, and Bogdan Vasilescu. Going farther together: The impact of social capital on sustained participation in open source. In 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), pages 688–699. IEEE, 2019.

[52] Jeffrey A Roberts, Il-Horn Hann, and Sandra A Slaughter. Understanding the motivations, participation, and performance of open source software developers: A longitudinal study of the apache projects. Management science, 52(7):984–999, 2006.

[53] Claude Sammut and Geoffrey I. Webb, editors. Leave-One-Out Cross-Validation. Springer US, Boston, MA, 2010.

[54] Andreas Schilling, Sven Laumer, and Tim Weitzel. Who will remain? an evaluation of actual person-job and person-team fit to predict developer retention in floss projects. In 2012 45th Hawaii International Conference on System Sciences, pages 3446–3455. IEEE, 2012.

[55] Pratyush N Sharma, John Hulland, and Sherae Daniel. Examining turnover in open source software projects using logistic hierarchical linear modeling approach. In IFIP International Conference on Open Source Systems, pages 331–337. Springer, 2012.

[56] Igor Steinmacher, Tayana Conte, Marco Aurélio Gerosa, and David Redmiles. Social barriers faced by newcomers placing their first contribution in open source software projects. In Proceedings of the 18th ACM conference on Computer supported cooperative work & social computing, pages 1379–1392, 2015.

[57] Igor Steinmacher, Igor Wiese, Ana Paula Chaves, and Marco Aurélio Gerosa. Why do newcomers abandon open source software projects? In 2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE), pages 25–32. IEEE, 2013.

[58] Yanmin Sun, Andrew KC Wong, and Mohamed S Kamel. Classification of imbalanced data: A review. International journal of pattern recognition and artificial intelligence, 23(04):687–719, 2009.

[59] Marat Valiev, Bogdan Vasilescu, and James Herbsleb. Ecosystem-level determinants of sustained activity in open-source projects: A case study of the pypi ecosystem. In Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pages 644–655, 2018.

[60] Diederik W van Liere. How shallow is a bug? why open source communities shorten the repair time of software defects. ICIS 2009 Proceedings, page 195, 2009.

[61] Yiqing Yu, Alexander Benlian, and Thomas Hess. An empirical study of volunteer members' perceived turnover in open source software projects. In 2012 45th Hawaii International Conference on System Sciences, pages 3396–3405. IEEE, 2012.

[62] Minghui Zhou and Audris Mockus. Developer fluency: Achieving true mastery in software projects. In Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering, pages 137–146, 2010.

[63] Minghui Zhou and Audris Mockus. What make long term contributors: Willingness and opportunity in oss community. In 2012 34th International Conference on Software Engineering (ICSE), pages 518–528. IEEE, 2012.

[64] Shurui Zhou, Bogdan Vasilescu, and Christian Kästner. How has forking changed in the last 20 years? a study of hard forks on github. In 2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE), pages 445–456. IEEE, 2020.